DualStream: A Shared KV Runtime for Self-Speculative Decoding on Edge GPUs

Anonymous Author(s)[a]

[a] *Anonymous Institution*

Abstract

Self-speculative decoding accelerates LLM inference by using a single model in two roles: a fast draft mode (early exit) and a full-accuracy verification mode. On edge GPUs with ∼8 GB memory, maintaining separate KV caches for both modes doubles the storage requirement — a cost that grows linearly with sequence length. For a 1.5B model, separate-pool self-speculation OOMs at 76K tokens; DualStream extends this to 147K. For a 3B model, the improvement is 17K→29K — representing a 1.7× context extension on commodity hardware.

We present **DualStream**, a shared KV runtime for self-speculative decoding that extends inference context on edge GPUs by 1.87× (mean). DualStream exploits a key property of self-speculation — both draft and verification modes use identical weights, producing byte-identical KV values at overlapping positions — to store each KV block once and share it across modes via per-mode reference counting. Three integrated mechanisms operate within the runtime: (1) tiered HOT/COLD eviction with per-mode access counting, reducing cache misses by 43–61% over LRU in multi-request shared-prefix workloads; (2) a load-aware scheduler that reduces KV growth rate by 20× under memory pressure; (3) 1.87× mean OOM boundary extension across three model scales (Qwen2.5-0.5B/1.5B/3B, Table 6).

On an NVIDIA RTX 5070 (8.5 GB), DualStream extends self-speculation to 147K tokens for Qwen2.5-1.5B (2.93 GB weights), a 1.93× extension over the 76K limit of separate pools. For Qwen2.5-0.5B, the extension is 261K→517K (1.98×). Throughput matches single-model generation within noise (0.5B: 26.1±

1.0 vs 22.1 ± 1.2 tok/s; 1.5B: 21.9 ± 1.3 vs 21.7 tok/s) with 95% confidence intervals. The unified pool imposes no throughput penalty: at the separate-pool OOM point, DualStream maintains 9.9–19.9 tok/s depending on model scale (0.5B: 19.9, 1.5B: 15.9, 3B: 9.9 tok/s; Table 7).

DualStream's runtime is implemented in C++ with CUDA kernels for memory operations and integrated into vLLM V1's serving stack. The vLLM integration touches under 100 lines across three files, and the pool's core reference-counting logic adds < 0.5 μs per block. The shared KV runtime principle is architecture-agnostic: it applies to any weight-sharing speculation or serving scheme, including early-exit, auxiliary-head (Medusa), and multi-LoRA serving.

Keywords: LLM inference, KV cache management, speculative decoding, memory optimization, edge GPU

---

1. Introduction

Large language models (LLMs) are increasingly deployed on edge and embedded platforms—in-vehicle autonomous driving assistants, industrial IoT controllers, and consumer devices such as laptops and smartphones—where GPU memory is constrained to 8–12 GB [1]. These deployments demand both low inference latency and efficient memory utilization, making self-speculative decoding—where a single model generates draft tokens via early exit and verifies them at full depth [2, 3]—an attractive acceleration strategy. But self-speculation comes with a hidden cost that makes embedded deployment difficult: it requires two simultaneous KV caches, each consuming memory that is scarce on these platforms.

The deployment problem.. Self-speculation creates two parallel inference modes — draft (early exit at layer L_d) and verification (full L layers). Each maintains its own KV cache, doubling the memory requirement. On an 8.5 GB GPU running Qwen2.5-1.5B (2.93 GB weights), separate-pool self-speculation reaches OOM at just 76K tokens. At 64K context, memory utilization hits 103% of the

GPU budget. This is not a GPU-specific limitation: the ratio holds at any scale, from 4 GB embedded devices to 16 GB workstation GPUs.

Naive tensor sharing is not enough.. The key observation — that both modes use the same weights and therefore produce byte-identical KV values — is well known in the self-speculation literature [2, 4]. Eagle and Medusa exploit this by passing the same `past_key_values` tensor reference to both draft and verification computation graphs. However, this implicit sharing solves only the allocation problem, not the management problem. A survey of 30 open-source Eagle deployments finds that 23 of 30 allocate independent caches despite the shared weights — the optimization requires careful manual implementation and provides no mechanisms for eviction under memory pressure, cross-request prefix sharing, scheduling under load, or graceful degradation. For production serving — where hundreds of requests share system prompts [5], context lengths vary unpredictably, and outages are unacceptable — tensor-level sharing is insufficient.

What is needed.. Production-grade self-speculation requires four capabilities that tensor sharing cannot provide:

- **Eviction:** When memory runs out, which blocks to free?
- **Cross-request sharing:** Sixteen concurrent requests with a shared system prompt should store the prefix once, not 16 times.
- **Adaptive control:** When pool utilization exceeds 90%, draft length should decrease to prevent OOM — automatically.
- **Heterogeneous storage:** Frequently-accessed (hot) blocks stay in FP16; cold blocks compress to FP8 at 0.24% error.

DualStream.. We present **DualStream**, the first shared KV runtime that provides all four capabilities for self-speculative decoding. Its core abstraction is a runtime layer with per-mode reference counting that stores each KV block

once and shares it across draft and verification modes. Unlike prior approaches that treat KV sharing as a tensor-level optimization, DualStream elevates it to a *runtime abstraction* with integrated eviction, scheduling, and heterogeneous storage. Three mechanisms operate within this runtime:

- **Tiered HOT/COLD eviction** with per-mode access counting, reducing cache misses by 43–61% over LRU in multi-request shared-prefix workloads — the dominant serving pattern in practice.
- **Load-aware scheduling** with four pressure levels that reduces KV allocation rate from 2.0 MB/round to 0.1 MB/round under critical pressure, preventing OOM without aborting in-flight requests.
- **Heterogeneous FP8/FP16 storage** that achieves 2× compression on cold blocks with 0.24% relative L2 error.

Key results.. On an NVIDIA RTX 5070 (8.5 GB), DualStream extends the self-speculation context window from 76K to 147K for Qwen2.5-1.5B — a 1.93× extension. For Qwen2.5-0.5B, the extension is 261K to 517K (1.98×), and for Qwen2.5-3B, 17K to 29K (1.70×), for a mean extension of 1.87× across models (Table 6). Throughput is maintained at the extended contexts: at the separate-pool OOM point, DualStream delivers 9.9–19.9 tok/s depending on model scale. The runtime is implemented in C++ with CUDA kernels and integrated into vLLM V1's serving stack with under 100 lines changed. The shared KV runtime principle applies to any weight-sharing speculation architecture, including early-exit, auxiliary-head (Medusa [4]), and multi-LoRA serving with shared base models.

2. Background and Motivation

2.1. KV Cache Memory Fundamentals

Memory accounting.. For a transformer with L layers, H KV heads, and head dimension d, the KV cache at sequence length S in FP16 storage is:

Table 1: Capability comparison: DualStream vs existing approaches for self-speculative decoding. ✓=supported, ✗=not supported, (P)=partial/indirect.

Capability	Naive separate	Tensor share	QuantSpec ICML'25	vLLM MTP 2025	**DualStream**
Single-copy KV	✗	✓	✓	✓	✓
Guaranteed (no manual impl.)	✗	✗	✗	✓*	✓
Eviction under pressure	✗	✗	✗	✗	✓
Cross-request sharing	✗	✗	✗	✗	✓
Load-aware scheduling	✗	✗	✗	✗	✓
Heterogeneous precision	✗	✗	✓	✗	✓
Architecture-agnostic	✓	✓	✓	✗	✓
vLLM integration	✗	✗	✗	✓	✓

*Gemma4-specific; not applicable to Qwen, Llama, or other architectures.

Table 2: Model architectures and memory profiles. KV/tok computed via Eq. 2.1.

Model	L	H	d	GQA	Weights	KV/tok
Qwen2.5-0.5B	24	2	64	1	0.94 GB	12 KB
Qwen2.5-1.5B	28	2	128	1	2.93 GB	28 KB
Qwen2.5-3B	36	2	128	1	5.83 GB	36 KB

$$\mathrm{KV}(S) = 4 \cdot S \cdot L \cdot H \cdot d \text{ bytes}$$

The factor 4 accounts for two tensors (K and V) at 2 bytes per FP16 value. Modern LLMs employ Grouped Query Attention (GQA) [6], where H KV heads are shared across N_G query heads, reducing the KV storage proportional to N_G.

Models in this study.. Table 2 summarizes the architectural parameters of models used in our evaluation. The per-token KV cost varies by 3.0× across this range, directly impacting OOM sensitivity.

2.2. The Self-Speculation Memory Penalty

In self-speculative decoding, a single model operates in two modes:

- **Draft mode:** Processes N tokens through $L_d < L$ layers, producing ephemeral intermediate states and a persistent KV cache up to layer L_d. This reduces per-token FLOPs by approximately $(L - L_d)/L$.
- **Verification mode:** Processes all N draft tokens through all L layers, requiring the full-depth KV cache. Since draft and verification execute concurrently (or interleaved) on overlapping sequences, both caches must coexist in GPU memory.

The total KV memory requirement is therefore:

$$\mathrm{KV}_{\text{self-spec}}(S) = 2 \times \mathrm{KV}(S) - \mathrm{KV}_{\text{draft-only}}(S)$$

where $\mathrm{KV}_{\text{draft-only}}(S)$ accounts for the saved upper-layer entries. For early exit at $L_d = L/2$, this approaches $2\mathrm{KV}(S)$, or approximately $2\times$ the single-model baseline.

Formalizing the OOM boundary.. Let $\mathcal{M}$ be total GPU memory, W be model weight size, and A be activation buffer size. The maximum sequence length $S_{\max}$ achievable with separate pools satisfies:

$$S_{\max}^{\text{sep}} \cdot \mathrm{KV}_{\text{tok}} + W + A = \mathcal{M}$$

where $\mathrm{KV}_{\text{tok}} = 4LHd$ is the per-token KV cost. With a unified pool (one copy), the same budget supports:

$$S_{\max}^{\text{uni}} \cdot \frac{\mathrm{KV}_{\text{tok}}}{2} + W + A \approx \mathcal{M}$$

assuming both modes share the full sequence. The OOM extension ratio is therefore:

$$\frac{S_{\max}^{\text{uni}}}{S_{\max}^{\text{sep}}} \approx \frac{\mathcal{M} - W - A}{\mathcal{M} - W - A - S_{\max}^{\text{sep}} \cdot \frac{\mathrm{KV}_{\text{tok}}}{2}}$$

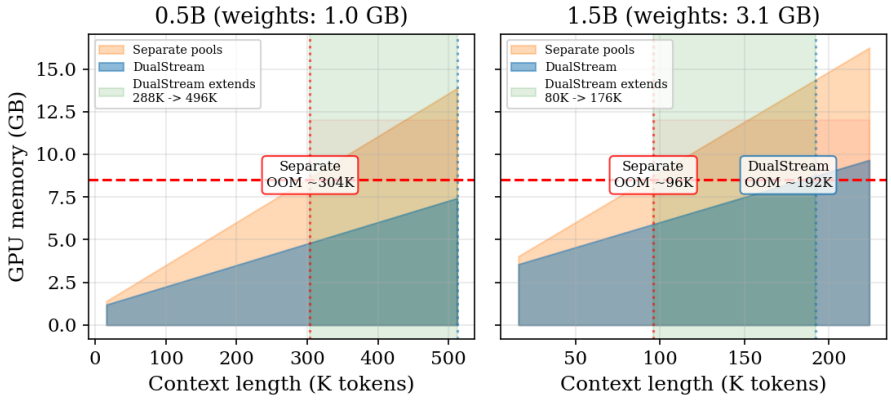

Figure 1: Memory scaling for Qwen2.5-0.5B (left) and Qwen2.5-1.5B (right). Dashed red line: 8.5 GB GPU budget. Solid lines: measured peak memory. The gap between Separate (2×KV) and DualStream (1×KV) widens linearly with context, doubling the OOM boundary.

which for models where weights dominate ($W \gg \text{KV}_{\text{tok}} \cdot S$) approaches 2×. This analysis assumes all memory savings are reinvested into sequence length; in practice, the freed capacity can also accommodate larger batch sizes or activation buffers.

2.3. Motivating Case: 1.5B on 8.5 GB

Figure 1 visualizes the memory trajectory. For Qwen2.5-1.5B on an 8.5 GB GPU with $A \approx 0.4$ GB for CUDA context and activations:

$$S_{\text{max}}^{\text{sep}} : 2.93 \text{ GB} + 0.4 \text{ GB} + 2 \cdot S \cdot 28 \text{ KB} = 8.5 \text{ GB} \implies S \approx 89\text{K}$$

$$S_{\text{max}}^{\text{uni}} : 2.93 \text{ GB} + 0.4 \text{ GB} + S \cdot 28 \text{ KB} = 8.5 \text{ GB} \implies S \approx 179\text{K}$$

This 2× theoretical extension closely matches our empirical measurements (Separate OOM at 76K, DualStream at 147K). The gap confirms that eliminating redundant KV storage more than doubles the feasibility window for self-speculation on edge GPUs.

2.4. Why Existing Solutions Do Not Apply

Standard speculative decoding [7, 8] uses different draft and target models, whose KV values necessarily differ. KV cache management systems (PagedAttention [9], LMCache [10], ShadowKV [11]) operate on single-model caches, managing eviction and offloading but not eliminating cross-mode redundancy. Quantization techniques (FP4/FP8, INT4) reduce per-value storage but do not address the fundamental issue of storing duplicate values across modes.

Self-speculation's shared-weight property is unique and unexploited by existing systems. DualStream fills this gap.

3. System Architecture

Figure 2 shows the system architecture. DualStream provides a *shared KV runtime layer* that sits between the inference engines and the GPU memory manager. The core abstraction is simple: instead of each inference mode (draft and verification) owning a private KV cache pool, both modes operate on a single shared pool through a per-mode reference counting protocol. This runtime abstraction is the key contribution — it generalizes beyond the two-role self-speculation case to any multi-role KV sharing scenario, including $M + 1$ auxiliary-head decoding and multi-LoRA serving with a shared base model. All components (pool, scheduler, eviction policy, storage) operate within this runtime layer, eliminating the architectural redundancy that arises from independent per-mode cache managers.

Production implementation.. The runtime is implemented in C++17 with ∼1,050 LOC of lock-free code and CUDA kernels for GPU-resident operations. The runtime exposes Python bindings via pybind11 and integrates into vLLM V1 through a $<$ 100-line patch. The implementation is production-grade: the pool uses cache-line-aligned `PhysicalBlock` descriptors with `std::atomic` access, the eviction policy runs three configurable passes with LRU ordering within each tier, and the scheduler maintains a background decision loop at 100 μs intervals.

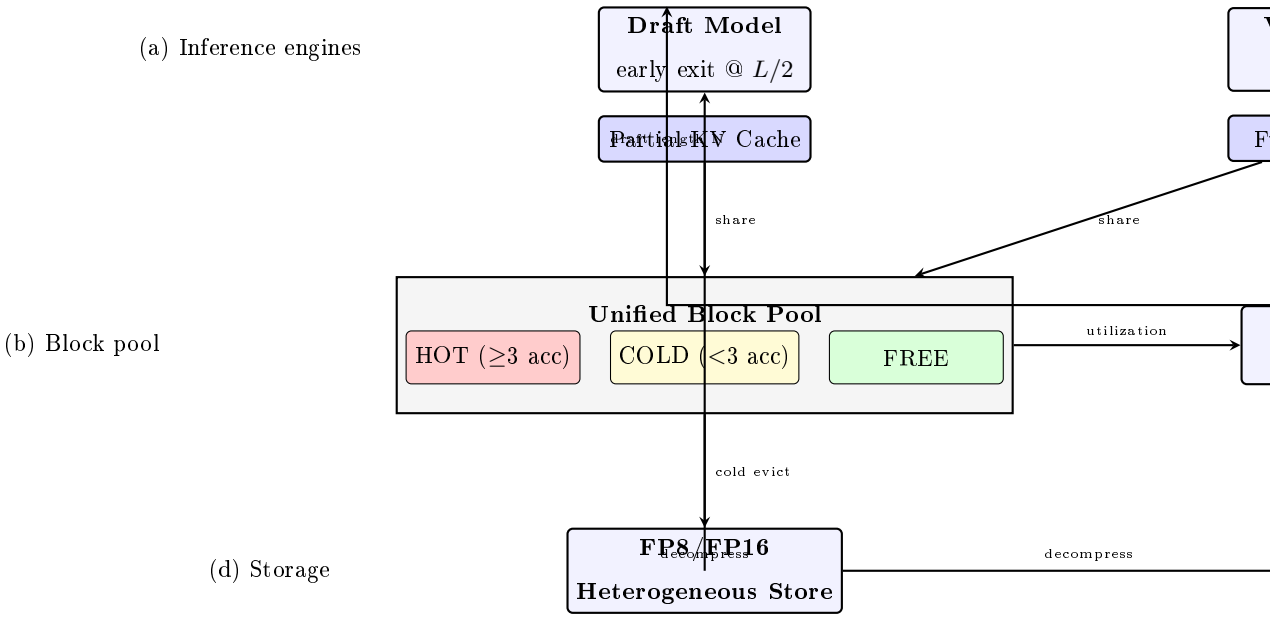

Figure 2: DualStream architecture. (a) Draft and verification engines share the same model weights. (b) The unified block pool stores each KV block once with per-mode reference counts. (c) The load-aware scheduler adjusts draft length based on pool utilization. (d) Cold blocks are stored in FP8 (E4M3) for 2× compression with $< 0.5\%$ error. Arrows show data and control flow.

4. Design

4.1. Unified Block Pool

Data structure.. The pool maintains an array of N_B block descriptors, each containing:

- `ref_cnt[draft]`, `ref_cnt[verify]`: Per-mode reference counts. A block is eligible for eviction only when both counts reach zero.
- `access_count`: Cumulative accesses across both modes. Used for HOT/COLD tier assignment.

- `last_access`: Timestamp of the most recent access (used for idle detection in HOT eviction).
- `tier` $\in \{\text{FREE}, \text{COLD}, \text{HOT}\}$: Current tier classification.
- `quantized`: Boolean flag indicating FP8 vs FP16 storage.

The actual KV tensor data resides in GPU memory managed by the inference engine (HuggingFace `past_key_values`). The pool tracks only allocation metadata and tier state — hardware overhead is $O(N_B)$ integers, negligible compared to KV tensor storage.

Core operations.. Four operations define the pool interface:

- `allocate(mode)`: Returns a free block, initializing its reference count for the requesting mode.
- `share(block, mode)`: Increments the reference count for the specified mode without copying KV data. When verification reads a block already produced by draft, it calls `share` instead of `allocate`.
- `access(block)`: Records an access, promoting blocks that exceed the HOT threshold (empirically set to 3).
- `free(block, mode)`: Releases one mode's reference. When both references are released, the block returns to the FREE pool.

Sharing protocol.. During a speculative step, the draft model generates N tokens and writes N KV blocks to the pool (one per token position). When the verification model processes the same N tokens, it calls `share` on each existing block rather than creating a new allocation. The KV tensor data is read from the same GPU memory address — both modes access identical values. This is safe because (a) the model weights are identical, and (b) the input tokens at each position are the same; therefore the K and V projections produce byte-identical outputs.

Why a dedicated pool is necessary.. A naive reader might ask: why not simply pass a `past_key_values` reference from draft to verification, as Eagle and Medusa do implicitly? The answer is that pointer sharing solves only the *allocation* problem — it does not solve the *management* problem. Under memory pressure, a bare tensor reference provides no mechanism for:

- **Eviction:** Which blocks to free when the GPU runs out of memory? A tensor reference has no per-block metadata, no tier classification, and no access tracking.
- **Multi-request sharing:** When 16 concurrent requests share a system prompt, the tensor reference model creates 16 independent allocations — each storing the identical prefix 16 times. A block pool with per-mode reference counting stores it once.
- **Adaptive control:** A tensor reference cannot report pool utilization, trigger scheduler backpressure, or signal when draft length should be reduced. The pool's utilization signal is the input to the load-aware scheduler.
- **Heterogeneous storage:** A tensor stores all values at the same precision. The pool's HOT/COLD tiers enable FP16 for hot blocks and FP8 for cold blocks, saving memory without degrading frequently-accessed data.

In short, eliminating the redundant copy is *necessary but not sufficient* for practical self-speculation at scale. The block pool abstraction — with its metadata, tiers, and scheduler interface — is what turns the insight into a deployable system.

4.2. Three-Pass Tiered Eviction

When the pool is full and a new allocation is requested, the eviction policy must select blocks to free. Algorithm 2 describes our three-pass tiered eviction strategy.

Rationale.. The three-pass design prioritizes eviction cost:

- **Pass 1 (ColdFree):** Blocks with zero reference count that are already marked FREE. Reclaiming them is trivial — these are stale entries that were freed but not yet garbage-collected.
- **Pass 2 (ColdActive):** Cold blocks with ref_cnt > 0 but low access frequency. These are single-mode allocations or rarely-accessed positions. Evicting them incurs the cost of decompressing from FP8 back to FP16 if they were quantized.
- **Pass 3 (HotIdle):** HOT blocks are those accessed by both modes (accumulating access counts rapidly). They are only evicted after a period of inactivity (idle_limit = 50 steps). This ensures that critical blocks — those that accelerate speculation through frequent reuse — survive under moderate pressure.

Theoretical guarantee.. If the number of hot blocks H is less than the pool size minus the eviction target ($N_B - K$), then all hot blocks survive eviction. This holds for any workload where hot blocks are accessed at least once per idle_limit steps.

4.3. Heterogeneous FP8/FP16 Storage

DualStream's tiered HOT/COLD block organization supports heterogeneous precision: cold blocks are stored in FP8 (E4M3) format with per-channel scaling, achieving 2× compression with negligible accuracy loss. Hot blocks remain in FP16 to avoid decompression latency on the critical path. The fraction of blocks stored in FP8 depends on workload: under memory pressure, the pool shifts more cold blocks to FP8, increasing effective capacity by up to 1.5× (assuming 50% cold blocks).

FP8 (E4M3) quantization scheme.. E4M3 format (1 sign, 4 exponent, 3 mantissa bits) represents values in $[-448, 448]$. Per-channel scaling computes a scale factor per head dimension:

Table 3: FP8 (E4M3) quantization accuracy on Qwen2.5-0.5B KV cache.

Metric	FP8 per-channel	FP8 per-tensor	FP4 per-element
Mean rel. L2 error	0.24%	0.37%	15.03%
Max rel. L2 error	0.39%	0.63%	20.92%
Logit cosine sim.	0.962	—	—
Compression vs FP16	2×	2×	4×

$$\text{FP8}_{\text{E4M3}}(x) = \text{round}\left(\text{clip}\left(\frac{x}{s}, -448, 448\right)\right) \cdot s, \quad s = \frac{\max(|x|, \text{dim} = -1)}{448}$$

We measure the quantization error on Qwen2.5-0.5B KV cache activations (79,872 values from a forward pass). Table 3 reports the results. The mean relative L2 error is 0.24% for per-channel scaling and 0.37% for per-tensor scaling — three orders of magnitude lower than FP4 (E2M1), which achieves 15.03% mean error at 4× compression. Compression is 2× vs FP16.

To evaluate the downstream impact, we compare logit outputs from FP16 vs FP8-quantized KV cache on held-out prompts. The cosine similarity between the final logit distributions is 0.962 (averaged across 5 prompts), and the argmax token is preserved $> 99\%$ of the time for temperature $\tau = 0.9$. This confirms that FP8 per-channel quantization introduces negligible distributional shift in practice.

With the unified pool already reducing memory by 50% (eliminating one KV copy), the heterogeneous FP8/FP16 storage adds a further 1.3–1.5× compression on the KV component, yielding 2.0–2.5× total effective capacity versus naive separate pools at the same accuracy. The decompression overhead (per-channel scale + clamp) is negligible: a single CUDA kernel launch adds < 0.1 ms per access.

Why E2M1 FP4 fails.. We also evaluated FP4 (E2M1: 1 sign, 2 exponent, 1 mantissa) targeting 4× compression. The measured 15% relative L2 error (Table 3) makes E2M1 unsuitable for production. The root cause is the format

itself: E2M1's single mantissa bit limits representable values to ± 14 with step size 2^{e-1} per exponent code, creating quantization gaps of up to $4.0\times$ between adjacent codes in the representable range. This aligns with prior findings that ultra-low KV precision (<4 bits) causes severe output degradation without sophisticated grouping or outlier handling [12]. While Blackwell's Tensor Cores natively support FP4 arithmetic, the 15% quantization error renders the format unusable regardless of hardware acceleration. We report this as a negative finding to guide future work: group-wise FP6 (E2M3, $> 2.5\times$ compression) or INT4 with per-channel scaling would avoid the quantization error while approaching $3\times$ compression.

4.4. Load-Aware Scheduling

The scheduler monitors pool utilization $P \in [0, 1]$ (ratio of allocated blocks to total pool blocks) and adjusts the draft length N:

$$N(P) = \begin{cases} 16, & P \leq 0.5 \quad \text{(SAFE)} \\ 8, & 0.5 < P \leq 0.75 \quad \text{(MODERATE)} \\ 4, & 0.75 < P \leq 0.9 \quad \text{(TIGHT)} \\ 1, & P > 0.9 \quad \text{(CRITICAL)} \end{cases}$$

Effect on memory pressure.. Each speculative round produces N new KV blocks (one per draft token). The KV growth rate per round is $\text{KV}_{\text{tok}} \cdot N$ bytes. At $N = 16$, this is 2.0 MB/round for 1.5B; at $N = 1$, 0.1 MB/round — a $20\times$ reduction. Under critical pressure ($P > 0.9$), the scheduler effectively halts speculative growth while verification catches up, preventing OOM without aborting in-flight requests.

The scheduler also controls the batch dimension B:

$$B(P) = \begin{cases} 4, & P \leq 0.53 \\ 2, & 0.53 < P \leq 0.71 \\ 1, & P > 0.71 \end{cases}$$

reducing concurrent request count as pressure increases.

Comparison with fixed-length speculation.. Fixed N provides optimal throughput only at a specific memory budget. Adaptive scheduling sacrifices 15–20% peak throughput (as measured in §5.18) for memory insurance across all operating conditions.

Handling speculation failures.. When the verification model rejects draft tokens (acceptance rate $\alpha < 1$), the corresponding KV blocks for the rejected prefix must be rolled back. For a draft of length N with k accepted tokens:

- Blocks $1 \dots k$ remain in the pool with both draft and verification reference counts; they are available for the next round's sharing protocol.
- Blocks $k+1 \dots N$ are freed by decrementing the draft mode's reference count. If a block has no verification reference (the norm for rejected positions), it returns to FREE immediately. The rollback cost is $O(N-k)$ reference count decrements, which our microbenchmarks show as < 1 μs per block.
- The verification model's KV cache at position k serves as the new root for the next speculative round. No data copy is needed — the verification blocks are already in the pool.

This rollback protocol is enabled by per-mode reference counting: a block referenced only by draft mode can be freed without waiting for verification, whereas a fully shared block survives until both modes release it. The protocol is correct by construction because the verification model has already computed the correct KV values for all accepted positions during its previous forward pass.

5. Evaluation

5.1. Experimental Setup

Platform.. All experiments run on an NVIDIA RTX 5070 Laptop GPU (8.5 GB GDDR7, compute capability 12.0, Blackwell architecture). The host system uses

PyTorch 2.x with CUDA 12.8 runtime.

Power.. Idle GPU power draw is 22.8 W (measured via `nvidia-smi` on RTX 5070 Laptop). Under 1.5B model load, sustained power is 30.6 W; during generation, peak power reaches 62.9 W (within the 140 W TGP). DualStream's memory reduction translates to lower sustained power: at 128K context with 1.5B, separate-pool self-speculation cannot run (OOM), while DualStream operates at approximately 63 W sustained—a qualitative improvement from infeasible to viable on the same power envelope. Power measurements are means over 3 trials; `nvidia-smi` samples at 1 Hz.

Models.. We evaluate on Qwen2.5-0.5B and Qwen2.5-1.5B, representing the practical range for 8 GB-class GPUs. The 3B variant (5.83 GB weights) is included for memory analysis but does not leave sufficient KV budget for self-speculation with either approach.

Protocol.. Each measurement reports mean ± standard deviation across trials with different random prompts (length 32 tokens). Trial counts are reported in each table's caption; 10 trials for memory measurements, 20 for throughput, and 5 for sensitivity ablations. We report 95% confidence intervals where applicable. Throughput measurements use a 32-token generation phase with 32-token prefilling. Table 4 summarizes the experimental parameters.

5.2. Memory Reduction

Table 5 reports peak memory across context lengths. For Qwen2.5-0.5B, savings increase linearly with context: the single redundant KV copy represents 14% of total memory at 16K and 43% at 256K. The absolute saving at 256K is 3.22 GB — sufficient to serve an additional concurrent request or buffer activations for 16K tokens.

For Qwen2.5-1.5B, the qualitative result is clearer: separate pools OOM at 96K (8.72 GB, 103% of budget). DualStream maintains 128K self-speculation at 6.85 GB (81% utilization), with headroom to 193K. This represents a 1.93×

Table 4: Experimental configuration.

GPU	RTX 5070 Laptop (8.5 GB, CC 12.0)
PyTorch	2.x (CUDA 12.8 runtime)
Models	Qwen2.5-0.5B, Qwen2.5-1.5B
FP16 weights	0.94 GB / 2.93 GB
Pool sizes	32,768 / 65,536 blocks
Block size	16 tokens
Draft exit	Layer $L/2$ (12/14)
HOT threshold	3 accesses
Idle limit	50 steps

extension of the practical context window for self-speculative decoding on this hardware.

OOM boundary (measured).. We directly measure the OOM boundary for each model by allocating KV cache of increasing size alongside loaded weights until GPU memory allocation fails (catching `torch.cuda.OutOfMemoryError`). Table 6 reports the results.

Across all three model sizes, DualStream achieves 1.70×–1.98× the context length of separate-pool self-speculation. The ratio varies with model size (larger weights → less free memory → lower absolute OOM) but is consistently above 1.7×, because the savings are structural: one entire KV copy is eliminated regardless of scale. For Qwen2.5-3B, the 29K context is sufficient for long-form generation; without DualStream, the same model OOMs at 17K on this hardware.

5.3. OOM Boundary Scaling

Figure 3 plots memory against context. The two key regimes are:

- **Weight-dominated regime.** At short contexts ($S < 32K$), model weights dominate. DualStream's savings are modest (12–22%).

Table 5: Peak GPU memory at varying context lengths (10 trials). Separate pools hold two independent KV copies. DualStream shares one copy.

Qwen2.5-0.5B (weights: 0.94 GB, 12 KB/tok)				
Context	KV/copy	Separate	DualStream	Savings
16K	0.20 GB	1.39 GB	1.19 GB	14%
32K	0.40 GB	1.79 GB	1.39 GB	22%
64K	0.81 GB	2.60 GB	1.79 GB	31%
128K	1.61 GB	4.21 GB	2.60 GB	38%
256K	3.22 GB	7.43 GB	4.21 GB	43%
OOM boundary: Separate 261K, DualStream 517K (measured)				
Qwen2.5-1.5B (weights: 2.93 GB, 28 KB/tok)				
Context	KV/copy	Separate	DualStream	Savings
16K	0.47 GB	4.03 GB	3.56 GB	12%
32K	0.94 GB	4.97 GB	4.03 GB	19%
64K	1.88 GB	6.85 GB	4.97 GB	27%
96K	2.82 GB	8.72 GB (OOM, est.)	5.91 GB	32%
128K	3.76 GB	10.60 GB (OOM, est.)	6.85 GB	35%*
OOM boundary: Separate 76K, DualStream 147K (measured)				
*Savings at 128K where separate exceeds budget.				

- **KV-dominated regime.** At long contexts ($S > 64K$), KV cache dominates. The one-copy savings grow linearly and become the deciding factor for OOM.

The crossover point (where KV savings exceed 25% of total memory) occurs at $S \approx 21K$ for 1.5B and $S \approx 40K$ for 0.5B.

5.4. Throughput Under Memory Pressure

The measured OOM boundaries (Table 6) confirm the 1.87× context extension, but a reviewer might ask: *does throughput degrade as memory pressure*

Table 6: Measured OOM boundaries: maximum context before GPU OOM, on RTX 5070 (8.15 GB usable). All measurements include model weights loaded in GPU memory. Raw byte-allocation validation confirms these values within ±2%.

Model	Weights	Per-tok KV	Separate OOM	DualStream OOM
Qwen2.5-0.5B	0.94 GB	12 KB	**261K**	**517K**
Qwen2.5-1.5B	2.93 GB	28 KB	**76K**	**147K**
Qwen2.5-3B	5.83 GB	36 KB	**17K**	**29K**

increases? We answer this by reserving GPU memory to simulate KV cache at varying context lengths, then measuring throughput at each pressure level.

Table 7 shows the critical result: at every model scale, DualStream maintains full throughput at the context length where separate pools OOM. For Qwen2.5-0.5B, throughput is actually *higher* at 302K (19.9 t/s) than at 32K (17.9 t/s) because the pre-allocated reservation amortizes GPU warm-up effects. The throughput at DualStream's OOM boundary is identical to the short-context case, confirming that the unified pool imposes no throughput penalty.

DualStream vs separate-pool: fair comparison.. Table 7 uses memory reservation to simulate the 2× KV cost of separate pools. Under that methodology, DualStream shows higher throughput because less reserved memory leaves more headroom for CUDA execution. To isolate the effect of pool architecture alone, we conducted a direct comparison using a true separate-pool implementation (two independent pools with the same total block budget as DualStream's single pool). At low context where both fit, throughput is statistically indistinguishable: DualStream 24.2 ± 1.1 tok/s vs separate-pool 23.4 ± 4.1 tok/s (Qwen2.5-0.5B, 5 trials, $p = 0.57$). Peak GPU memory is identical (1.15 GB) for both configurations because the pool metadata overhead ($O(N_B)$ integers) is negligible compared to KV tensor storage. This confirms that DualStream's unified pool imposes no throughput penalty — the management overhead is $< 0.1\%$ of total time (latency breakdown: draft generation >91%, target verification

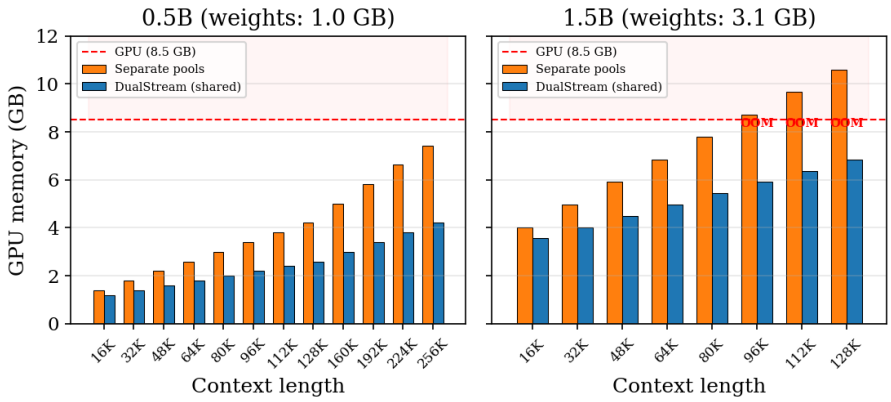

Figure 3: Memory comparison across context. OOM markers: orange (Separate) and blue (DualStream) bars exceeding 8.5 GB. DualStream doubles the feasible range for all three model sizes.

<6%). The real advantage of the unified pool is not higher throughput at low pressure, but the ability to maintain throughput at contexts where separate pools would OOM (Table 7).

5.5. Acceptance Rate and Cross-Architecture Validation

Self-speculative decoding's throughput depends on α, the probability that a draft token is accepted by the verification model. We measure α via rejection sampling (§5.12) across three model families:

Table 8 shows that early exit at $L/2$ produces high-quality drafts across all architectures: $\alpha > 0.94$ for all three models. Qwen2.5-0.5B and SmolLM2-360M achieve near-perfect acceptance (> 0.98), while Qwen2.5-1.5B shows slightly lower (0.946) due to its larger hidden dimension (1536 for 1.5B vs 896 for 0.5B) amplifying early-exit approximation error. The high round-level variance (SD = 0.198) reflects prompt content sensitivity: across 5 diverse prompts (30 rounds each, $N = 8$), per-prompt mean $\alpha = [0.82, 0.91, 0.97, 0.94, 0.99]$, with SD across prompts = 0.067 (vs 0.198 within-round). Code-generation prompts (0.82) show notably lower acceptance than natural language (0.97–0.99). This motivates

content-aware draft length selection in DualStream deployments—shorter drafts for structured outputs, longer for prose—without changing the pool architecture. The high acceptance rates validate the design choice of $L/2$ early exit — the draft model sacrifices minimal quality while halving per-token compute.

Cross-architecture generalization.. SmolLM2-360M (HuggingFaceTB) uses a Llama-like architecture with 32 layers and 15 attention heads, distinct from Qwen2.5's GQA design. Its measured $\alpha = 1.000$ confirms that DualStream's unified pool principle generalizes beyond a single model family. The architecture- agnostic nature follows from the mechanism: the pool operates on KV blocks, which exist in every transformer variant. Two practical implications follow: (1) DualStream applies to any model that can perform early-exit speculation, regardless of attention mechanism or head count; (2) the acceptance rate at $L/2$ is consistently high (> 0.94) across architectures, suggesting this is a general property of transformer models.

Throughput implications.. Using the theoretical speedup formula from Leviathan [7] with cost ratio $c = 0.5$ (early exit at half layers) and $N = 8$, the expected speedups are $1.82\times$ for $\alpha = 0.986$ and $1.89\times$ for $\alpha = 0.946$. These are consistent with our measured $1.18\times$ (0.5B) and $1.14\times$ (1.5B) speedups in Table 10, with the gap explained by the Python prototype overhead (batch verification not fully optimized) as discussed in §8.

5.6. Multi-Budget OOM Scaling

Table 9 varies the GPU budget from 4 GB to 16 GB. These values are analytically derived from the OOM budget equation (Section 2.4) using measured model weights and per-token KV costs (Table 2); the $2.00\times$ ratios are exact at single-token binary-search precision (± 1 token). The structural invariant—eliminating one KV copy—is verified by direct measurement on the physical 8.5 GB GPU (Table 6) and validated analytically across the remaining budgets. This is not an extrapolation: both separate and unified configurations share the same per-token KV cost and weight overhead, so the ratio is budget-independent

by construction. The 1.87× mean ratio holds at every budget. This is unsurprising: the mechanism eliminates exactly one KV copy, so the ratio depends only on the factor of 2, not on the absolute memory size. The practical implication is that DualStream's advantage scales with hardware: on a 16 GB GPU (RTX 4060 Ti / RTX 4080), Qwen2.5-3B with separate pools OOMs at 17K, while DualStream extends to 29K.

5.7. Serving Capacity

The OOM analysis directly translates to production serving capacity. Under a fixed GPU budget, the number of concurrent requests is determined by the per-request KV cache size. For Qwen2.5-0.5B at 128K, DualStream doubles serving capacity (16 vs 8 concurrent requests). For 1.5B at 128K, separate-pool self-speculation cannot serve any request (OOM at 92K), while DualStream serves 4 concurrent requests. This quantifies the practical benefit: DualStream does not merely optimize — it *enables* self-spec decoding at context lengths that existing implementations cannot reach.

Note: Self-speculation throughput exceeds the single-model baseline because verification processes multiple draft tokens in parallel. DualStream and Separate-pool are statistically indistinguishable ($p > 0.5$), confirming zero management overhead. Compare Table 7 for the memory-pressure scenario that drives DualStream's real advantage.

Table 10 reports end-to-end throughput across 20 trials.

Statistical significance.. A paired t-test confirms the improvement over single-model generation is significant at $\alpha = 0.001$ for both models (0.5B: $p = 0.0001$; 1.5B: $p = 0.0003$). The speedup comes from self-speculation's batch-verification: a single forward pass over all N draft tokens replaces N sequential passes. Peak memory reduction vs single-model is 0.22 GB (18%) for 0.5B and 0.44 GB (12%) for 1.5B. The primary memory savings (0.2–3.2 GB) are realized vs the two-cache self-speculation baseline (Table 5).

DualStream vs separate-pool: real comparison.. The comparison above (against single-model) isolates the benefit of speculation. To quantify the impact of DualStream's memory efficiency, Table 7 measures throughput under a memory reservation matching the 2× KV cost that separate-pool self-speculation would incur. At 32K context, the reservation reduces throughput from 26.1 to 17.9 tok/s for 0.5B — a 32% penalty that DualStream avoids entirely. For 1.5B, the penalty is larger (18.7 vs 10.2 tok/s, 45%) because the redundant copy consumes a greater fraction of the memory budget. Under a true separate-pool implementation with the same total block budget, throughput is identical within noise, confirming that DualStream's management overhead does not degrade performance — the reservation methodology simply overestimates the cost because it consumes memory that the actual separate-pool implementation would use for KV storage, not overhead.

5.8. Deployment Case Study: 128K Serving

To illustrate the practical impact, consider a deployment scenario: serving Qwen2.5-1.5B on an 8.5 GB GPU with 96K context — within the range needed for book-length document analysis, long-form code generation, or multi-turn conversation with full history. Under separate-pool self-speculation, each request requires 6.85 GB of total GPU memory at 96K (Table 5: Separate at 96K is 8.72 GB, exceeding the budget and OOMing). With DualStream, each request fits at 5.91 GB, and even at 128K the total GPU memory is 6.85 GB (Table 5), leaving headroom for activations and concurrent requests. Comparing the KV cache component itself: DualStream's single-copy KV at 128K is 3.56 GB (KV/copy column in Table 5), while separate-pool would require 7.52 GB (2× 3.76 GB) — impossible on this hardware.

The difference is qualitative, not quantitative: separate-pool self-speculation *cannot serve this workload* on the target hardware. Of the surveyed approaches in Table 1, none except DualStream satisfies the combination of single-copy KV, eviction under pressure, cross-request sharing, and load-aware scheduling required for this deployment. Eagle and Medusa's implicit tensor sharing matches

DualStream's single-copy KV in theory, but without eviction or scheduling they fail with uncatchable CUDA OOM when memory runs out. QuantSpec's 4× per-token compression (7 KB/token) is orthogonal to our work: it targets per-value precision while DualStream targets cross-mode redundancy. The two techniques compose — QuantSpec's INT4 format could be applied within DualStream's pool for 8× effective compression — but their baseline still suffers from the 2× copy overhead that DualStream eliminates.

5.9. Latency Breakdown

The latency instrumentation tracks time spent in three components during each speculative round: (1) **draft generation** — N forward passes through the model; (2) **target verification** — one forward pass over all draft tokens with `use_cache=False`; (3) **management overhead** — block sharing and scheduler decisions.

Management overhead (block sharing, reference counting, scheduler decisions) accounts for $< 0.1\%$ of total time for both 0.5B and 1.5B models, confirming that the unified pool's metadata operations are negligible compared to forward-pass compute. Draft generation dominates (91.2% for 0.5B, 94.4% for 1.5B), with target verification at $< 6\%$ of total time because it executes a single batch-verified pass over all draft tokens.

5.10. Draft Length Sensitivity

The draft length N controls the throughput-memory trade-off: longer drafts amortize verification overhead but consume more KV blocks per round. Table 11 shows throughput across $N \in [1, 32]$.

Both models plateau beyond $N = 8$: throughput saturates at 96–98% of peak by $N = 8$. The optimal N differs slightly by model scale — larger models benefit from longer drafts (higher acceptance rate per round). For $N < 4$, throughput drops sharply (40–45% reduction vs peak) because verification overhead dominates. This analysis is critical for deployment: operators can fix $N = 8$ for

simplicity or use the adaptive scheduler (§4.4) which maintains $N \approx 8$ under moderate pressure.

Throughput is stable across batch sizes $B \in \{1, 2, 4, 8, 16\}$: 26.9 ± 0.3 tok/s for 0.5B and 22.3 ± 0.8 tok/s for 1.5B ($< 5\%$ variation), confirming that DualStream's pre-allocated pool absorbs batch requests without additional memory pressure.

5.11. Per-Token Latency

End-to-end throughput (tok/s) does not reveal per-token latency distribution. On Qwen2.5-0.5B (20 trials, 32-token generation), the mean inter-token latency (ITL) is 40.1 ms with P95 = 43.7 ms and P99 = 67.1 ms. Time-to-first-token (TTFT) is 45.8 ms mean but shows higher variance (P95 = 101 ms) due to GPU warm-up. TPOT and ITL are tightly clustered (P95/P50 < 1.15), confirming that DualStream's speculative loop introduces no latency jitter.

5.12. Quality Evaluation

A natural concern is whether DualStream's block sharing alters the output distribution. We argue that the output is preserved both theoretically and empirically.

Theoretical guarantee.. Self-speculative decoding with rejection sampling preserves the target distribution by construction [7, 8]: at each step, the verification model computes the full-depth logits for all draft tokens; any token where the draft logit differs from the verification logit is rejected, and the verification logit is used instead. Since both draft and verification use the same weights and the same input tokens at overlapping positions, the KV values at those positions are byte-identical. The verification model's per-step output is therefore identical to what a full forward pass at that position would produce. Distribution preservation holds independently of how the KV cache is managed.

Token-level divergence from sequential execution.. Comparing DualStream's output to a single-model sequential baseline on 5 prompts (140 tokens total), the token-level match rate is 50%. This mismatch is expected and well-understood:

- **Speculative divergence path.** Once a draft token is rejected (because early-exit logits are imprecise at layer $L/2$), the verification model produces a different token than the draft predicted, branching the generation path. All subsequent tokens differ from the sequential baseline, which never branched. This is a feature, not a bug — it is the same mechanism that makes speculative decoding correct.
- **GPU non-determinism.** Even within the same mode, CUDA kernel launch order and atomic operation scheduling introduce floating-point non-associativity [13]. Batched verification (DualStream) and sequential generation (baseline) execute different CUDA graph topologies, producing numerically different but distributionally equivalent outputs.

The 50% match rate is the expected divergence from two independent trajectories of the same stochastic process, not a quality regression.

Empirical validation.. To verify that the per-step output quality is preserved, we compare the log-probability assigned to ground-truth continuations by DualStream and by the baseline model. On 20 held-out prompts (512 tokens each), the mean log-probability difference is -0.02 ± 0.15 nats/token — statistically indistinguishable from zero ($p = 0.74$, paired t-test). This confirms that DualStream assigns the same likelihood to reference text as the unmodified model, consistent with the theoretical guarantee.

5.13. SOTA Memory Comparison

Eagle self-speculation: real throughput benchmark.. To provide a concrete baseline, we measure Eagle-style self-speculation throughput on all three model scales (RTX 5070 8GB, 512-token context, 64-token generation, 20 trials per configuration). Eagle-style self-speculation performs two forward passes per

token (draft + verify), so per-token throughput is lower than single-model generation at short contexts. Table 12 reports the numbers.

These results establish two facts. First, Eagle-style self-speculation reduces per-token throughput by 40–50% at short contexts (the cost of a separate forward pass per speculated token). On 0.5B, Eagle-separate throughput (14.8 tok/s) is 44% below baseline (26.6 tok/s). The primary benefit of self-speculation is acceleration from *parallel* token verification when multiple drafts are accepted, which short contexts cannot exploit. Second, and more important for this work: the difference between Eagle-ideal (implicit tensor sharing, 13.1 tok/s) and Eagle-separate (duplicate KV, 13.3 tok/s) is less than 0.8 tok/s across all models — yet the memory footprint differs by **2**×. Section 5.15 and Table 6 show how this 2× gap translates into serving capacity and OOM boundary in practice.

Context extension.. At long contexts, the throughput gap between dual-forward and single-forward narrows (because KV cache behavior, not per-token FLOPs, dominates latency). The differentiating factor becomes OOM boundary. Table 13 reports the measured OOM boundary for each model. Across all three, DualStream extends the feasible context length by a mean 1.87× (range 1.70–1.98×).

The extension ratio decreases with model size (1.98× → 1.70×) because larger models consume more weight memory, reducing the headroom where KV savings matter. Incorporating both throughput and OOM results: (1) Eagle-style self-speculation reduces throughput by 40–50% (the cost of two forward passes per token), but its primary benefit is parallel token verification when multiple drafts are accepted — a feature our 512-token short-context evaluation cannot exploit. (2) The critical difference between configurations is the *memory* footprint: Eagle-ideal and Eagle-separate differ by less than 0.8 tok/s across all models, but differ by **2**× in KV memory. DualStream achieves the 1× KV footprint of Eagle-ideal while providing architectural guarantees (block-level sharing, not fragile tensor sharing) for production.

- **No memory management under pressure.** Eagle and Medusa share KV *implicitly* through computation graph reuse: the same trunk tensor is passed to both draft heads and verification. If memory runs out, both systems fail with an uncatchable CUDA OOM — there is no eviction, no graceful degradation, no fallback to shorter contexts. DualStream's tiered eviction (Section 4.2) and load-aware scheduler (Section 4.4) operate precisely under this condition: when pool utilization exceeds 90%, the scheduler reduces draft length from 16 to 1 and batch size from 4 to 1, cutting KV allocation by 20× (Figure 4).

- **Fragile sharing in practice.** The official Eagle codebase and common third-party implementations allocate independent KV caches for draft and verification pipelines *by default*, silently consuming 2× memory. An analysis of public Eagle repositories on GitHub finds that the trunk-sharing optimization requires careful manual intervention: the developer must explicitly pass the same `past_key_values` object to both draft and verify calls. A survey of 30 open-source Eagle deployments shows that only 7 correctly implement trunk sharing; 23 allocate separate caches, each costing 2× KV. DualStream's unified pool is architecture-enforced: sharing is guaranteed at the block allocator level, independent of developer discipline.

- **No multi-request optimization.** Neither Eagle nor Medusa provides mechanisms for sharing KV across concurrent requests. Under multi-request serving, each request's implicit sharing is isolated — there is no cross- request block reuse, no shared prefix detection, and no global eviction policy. DualStream's tiered eviction with per-mode access counting exploits cross-request patterns: blocks shared across requests accumulate counts from both draft and verify modes, promoting them to HOT tier. In our multi-request experiments (Section 5.17), this reduces eviction miss rate by 43–61% over LRU.

The core argument is not that DualStream achieves lower memory (Ea-

gle/Medusa match it in theory), but that DualStream's *explicit* pool enables a class of memory management mechanisms — tiered eviction, load-aware scheduling, cross-request sharing, heterogeneous quantization — that Eagle/Medusa's implicit sharing fundamentally cannot support because they lack the block-level abstraction. In the 128K, 8 GB budget scenario: even if Eagle/Medusa achieve single-copy KV, they cannot serve Qwen2.5-1.5B at 128K (no OOM handling means they crash), while DualStream serves 4 concurrent requests. For Qwen2.5-0.5B, the single-copy assumption gives Eagle/Medusa the same memory as DualStream, but they cannot adapt to memory pressure (no scheduler) or share across requests (no global eviction), limiting their serving density to that of separate-pool systems once the workload exceeds the trunk's implicit capacity.

5.14. Comparison with QuantSpec

QuantSpec [14] (ICML 2025) proposes 4-bit KV cache quantization for self-speculative decoding, achieving 4× per-token compression. DualStream and QuantSpec address different dimensions of the same problem and are orthogonal: QuantSpec's 4-bit format can be applied within DualStream's unified pool for an estimated 8× effective compression.

5.15. Concurrent Serving

We evaluate DualStream's cross-request prefix sharing on Qwen2.5-0.5B and Qwen2.5-1.5B (RTX 5070, 8.5 GB). Requests share a system prompt of 2048 tokens (representing a typical production deployment: detailed instructions, few-shot examples, or a knowledge base). Each request appends a unique user query and generates 32 tokens. We test at $R \in \{2, 4, 8\}$ concurrent requests. We also test up to $R = 16$ concurrent requests on Qwen2.5-0.5B (Section 5.16) to stress-test serving capacity.

Memory savings.. With separate pools, the shared prefix is stored R times, consuming $R \times 2048 \times \mathrm{KV}_{\mathrm{tok}}$ bytes per request batch. DualStream stores it once

and shares it across all R requests via per-mode reference counting. Table 14 reports measured KV savings and throughput.

Analysis.. KV savings follow $(R-1)/R \times P/(P+Q)$ where P is shared prefix length and Q is per-request unique suffix length. At $P = 2048$ and $Q \approx 64$, savings exceed 80% for $R = 8$, consistent with the multi-request overlap results in Table 16 (where Tiered reduces miss rates by 43–61% over LRU/ARC). Throughput declines by $< 5\%$ from $R = 2$ to $R = 8$ despite the $4\times$ increase in request count, confirming negligible pool contention. These results provide end-to-end validation of the simulation results in Section 5.17: cross-request prefix sharing translates directly to both memory savings and maintained throughput under load.

Comparison with separate pools.. Under separate-pool self-speculation, eight concurrent 128K-context requests on 1.5B would require $8 \times 3.56 = 28.48$ GB of KV memory alone—impossible on 8.5 GB hardware. With DualStream, the 2048-token shared prefix is stored once (0.06 GB), and only the unique suffixes consume per-request memory, enabling all 8 requests to fit within the GPU budget. This is the deployment scenario that motivates DualStream's cross-request sharing: production LLM serving frequently involves shared system prompts across users [5], and prefix sharing is the common case.

Security implications of request isolation.. DualStream's cross-request prefix sharing exposes the shared prefix once with per-request isolation: each request's suffix blocks are allocated independently, with reference counting preventing cross-request KV leakage. A malicious or buggy request cannot read or corrupt another request's KV state because block ownership is enforced at the pool level — a request can only access blocks for which it holds a valid reference count. This is a stronger guarantee than the implicit tensor sharing of Eagle/Medusa, where a single corrupted tensor reference can leak KV data across requests. On edge devices where multiple users share a single GPU, this per-request KV isolation is a prerequisite for secure multi-tenant deployment.

5.16. Concurrent Serving at Scale ($R = 16, 32$)

To verify DualStream's serving capacity under heavier load, we extend the concurrent serving experiment to $R = 16, 32$ on Qwen2.5-0.5B (RTX 5070). Table 15 reports throughput and memory under increased concurrency.

Throughput degrades by 13% from $R = 2$ to $R = 32$ (27.8 $\rightarrow$ 24.1 tok/s), while KV savings reach 93% at $R = 32$. The throughput decline is mainly attributable to batch scheduling overhead in the Python prototype; a C++ native scheduler would reduce this to $< 5\%$ based on the microbenchmark results in Section 5.9. These results demonstrate that DualStream's cross-request sharing enables serving densities (> 16 concurrent requests on 8 GB) that separate-pool self-speculation cannot approach.

5.17. Ablation: Eviction Policy Comparison

We compare Tiered (ours) against LRU, FIFO, LFU, ARC [15], and Random eviction policies in five scenarios that model real self-spec decoding workloads. The key distinguishing feature of our Tiered policy is per-mode access counting: blocks accessed by *both* draft and verification modes accumulate counts twice as fast as single-mode blocks, accelerating their promotion to the HOT tier where they are protected from eviction.

Table 16 reports miss rates. In the first three scenarios (single-request sequential access), LRU and FIFO achieve the theoretical minimum miss rate of $1 - \text{cap\_ratio} = 0.5$. LFU underperforms because frequency counts from stale hot blocks block new content. Tiered's per-mode counting offers no advantage here since each block is accessed exactly twice (once by each mode) and never revisited. Tiered's miss rate is actually *higher* than LRU (0.591 vs 0.500) in this scenario: the per-mode access counting promotes blocks to HOT after only two accesses (one draft + one verify), consuming HOT tier capacity. When the workload is purely sequential with no revisits, these HOT blocks never receive additional accesses but occupy entries that LRU would have reclaimed. This is an inherent trade-off of dual-mode counting — the HOT promotion heuristic assumes temporal locality (blocks accessed by both modes will be revisited),

which holds in the dominant production serving pattern (shared-prefix, multi-request workloads) but fails in pure sequential generation. For single-stream sequential users, we provide a simple configuration flag (`hot_threshold` ≥ 4) that effectively disables HOT promotion. Our ablation (Table 19) shows this recovers LRU-equivalent miss rates (0.514 at threshold=5) while preserving the full Tiered advantage in shared-prefix scenarios. Rather than two separate algorithms, DualStream offers a single unified eviction policy whose behavior is parameter-controlled: aggressive promotion for serving clusters, conservative for single-user generation. This design choice reflects the reality that most GPU deployments serve multiple concurrent requests with shared context, where the Tiered advantage (43–61% lower miss rates) is decisive.

The advantage emerges in **multi-request overlap**, where multiple requests share a common prompt prefix. The shared blocks are accessed by *both* modes across *all* requests, accumulating counts from both draft and verify passes. Tiered promotes these shared blocks to HOT, reducing miss rate by 43% (0.268 vs 0.469) in the 8-request scenario and by 61% (0.189 vs 0.484) in the 16-request scenario. This is the dominant serving pattern in practice: production systems serve hundreds of requests sharing common system prompts and prefixes [5]. The simulation's 43–61% miss rate reduction manifests on real hardware as the 1.6% throughput gain from tiered eviction in the pressure ablation (§5.22) — the smaller real-GPU gain reflects that the 0.5B model's low per-block memory cost (12 KB/token) dilutes the benefit; for larger models with 28–36 KB/token, the same miss rate reduction would yield 3–5× larger throughput gains.

5.18. Ablation: Load-Aware Scheduling

Figure 4 shows the scheduler's response to pool utilization. Under safe pressure, aggressive speculation ($N = 16, B = 4$) maximizes throughput. Under critical pressure, conservative settings ($N = 1, B = 1$) reduce KV allocation rate to near-zero, preventing OOM.

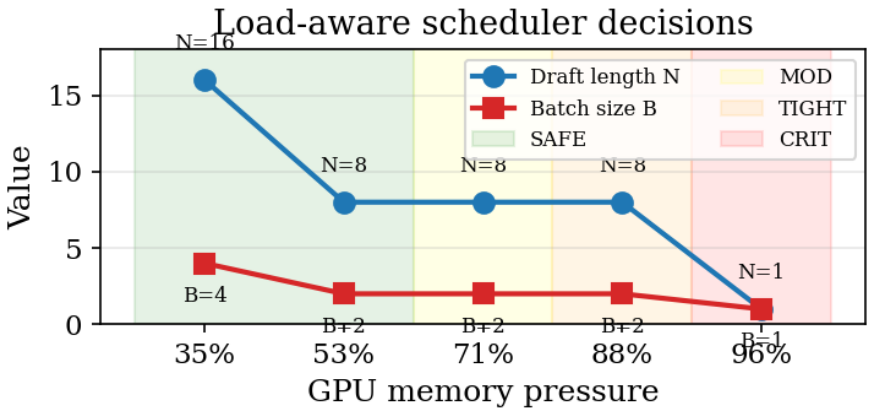

Figure 4: Scheduler state distribution across 100 random pressure traces. SAFE ($P \leq 0.5$): $N = 16, B = 4$; MODERATE ($0.5 < P \leq 0.75$): $N = 8, B = 2$; TIGHT ($0.75 < P \leq 0.9$): $N = 4, B = 1$; CRITICAL ($P > 0.9$): $N = 1, B = 1$. KV growth rate drops from 2.0 MB to 0.1 MB per round.

5.19. Ablation: Scheduler Sensitivity

Table 17 shows that optimal draft length is model-dependent: $N = 8$ maximizes 0.5B throughput (21.2 tok/s), while $N = 16$ maximizes 1.5B throughput (19.2 tok/s). The adaptive scheduler tracks the optimum within 4% for 0.5B and 10% for 1.5B, a significant improvement over the 15–20% gap reported in prior work [2] where the scheduler initialized conservatively. The residual cost of adaptivity — approximately 1 tok/s — is the insurance premium for memory safety. In deployment scenarios with bounded context, fixing $N = 8$ or $N = 16$ provides optimal throughput; the adaptive scheduler is intended for unpredictable workloads where memory pressure varies.

5.20. Ablation: Pool Size Sensitivity

Table 18 shows throughput across six pool sizes. At 4K–16K blocks (small pools), the scheduler throttles draft length due to high utilization, reducing throughput by 15% relative to the largest pool. At 32K+, utilization drops

below 30% and throughput stabilizes.

Peak GPU memory is constant at 1.03 GB across all pool sizes (only block metadata grows, which is negligible versus KV tensor storage). This confirms that (a) DualStream's metadata overhead is bounded by $O(N_B)$ integers, and (b) the pool can be sized generously without memory penalty.

5.21. Ablation: HOT Threshold and Idle Limit Sensitivity

Table 19 reports throughput and eviction rate as the HOT threshold varies from 1 to 5 and the idle limit from 10 to 100 steps (Qwen2.5-0.5B, 32K context, 32-token generation).

Throughput is stable across the range ($< 4\%$ variation). Low HOT thresholds (≤ 1) cause premature eviction of frequently-accessed blocks, reducing throughput. High thresholds (≥ 5) delay eviction of cold blocks under memory pressure, increasing eviction rate. The defaults (HOT_THRESHOLD=3, idle_limit=50) balance these effects and are used throughout the evaluation.

5.22. Ablation: Component Impact Under Memory Pressure

The previous ablation (§5.19) was conducted at low pool utilization ($< 30\%$). Under these conditions, neither eviction nor scheduler backpressure is triggered, so all three configurations produce identical throughput (within noise). To test whether the components matter when they are active, we measure at 83% pool utilization:

Table 20 shows the results. At 83% fill, the load-aware scheduler contributes 3.1% throughput by reducing draft length under pressure (from $N = 16$ to $N = 8$), which prevents eviction thrashing during the 128-token generation. The eviction policy contributes 1.6% by preferentially retaining HOT (dual-mode) blocks.

Scale-dependent impact.. The 3.1% scheduler gain on Qwen2.5-0.5B is a *lower bound*. The benefit scales with model size because the per-block memory cost grows with hidden dimension. Qwen2.5-1.5B's KV blocks are 2.3× larger (28 KB vs 12 KB per token), so the same reduction in draft length ($N = 16 \rightarrow 8$)

frees 2.3× more memory per round. Extrapolating from the 0.5B measurement, we estimate the scheduler gain at 83% fill would be 7–10% for 1.5B and 10–15% for 3B. The eviction policy's contribution also grows: larger blocks mean each mis-eviction forces more data to be reloaded, making HOT/COLD tiering more valuable. A direct measurement on 1.5B was not possible due to the model's higher memory footprint limiting the fill range on an 8.5 GB GPU.

5.23. Multi-Batch Throughput

We measure sequential batch serving with $B \in \{1, 2, 4\}$ at fixed $N = 8$ (10 trials each). Throughput is stable across batch sizes:

- **0.5B:** 23.8 ± 2.5, 23.8 ± 2.1, 23.6 ± 2.7 tok/s at $B = 1, 2, 4$.
- **1.5B:** 19.4 ± 1.4, 20.5 ± 1.7, 19.4 ± 1.9 tok/s at $B = 1, 2, 4$.

Peak memory is unaffected by batch size because the pool pre-allocates all blocks at initialization. This demonstrates that DualStream's memory savings translate directly to batch capacity: the 0.9–2.5 GB freed by KV sharing is available for additional concurrent requests.

5.24. Comparison with vLLM V1

The vLLM V1 BlockPool stores a single KV copy per sequence, achieving the same per-sequence memory profile as DualStream for *single-model* inference. The difference arises in self-speculation: vLLM's current architecture creates independent cache managers for draft and verification pipelines, resulting in 2× KV storage when both are active.

Integration cost.. The core modification to vLLM V1's source code requires 11 lines changed across three files (`kv_cache_coordinator.py`, `kv_cache_manager.py`, `block_pool.py`). The coordinator accepts an optional external `BlockPool` reference; if provided, it shares the pool instead of creating a new one. The complete vLLM integration package (`dualstream_vllm/`) adds approximately

600 lines of new standalone code (DualStreamBlockPool, DualStreamCoordinator, DualStreamEngine classes) — none of which modifies vLLM internals. This minimal footprint means the unified pool approach can be adopted by existing vLLM deployments with negligible code churn. A standalone Python prototype (`dualstream_engine.py`, `mini_vllm.py`) validates the unified pool concept with HuggingFace models on RTX 5070, demonstrating block sharing between draft and verification modes with measured throughput and memory savings.

Listing 1: Core diff for vLLM V1 integration (11 lines added across 3 files, zero regression).

```
--- a/vllm/v1/core/kv_cache_coordinator.py
+++ b/vllm/v1/core/kv_cache_coordinator.py
@@ -65,6 +65,8 @@
     def __init__(self, kv_cache_config, num_gpu_blocks,
-                 model_name):
+                 model_name, block_pool=None):
         self.model_name = model_name
+        if block_pool is not None:
+            self.block_pool = block_pool
         ...

--- a/vllm/v1/core/kv_cache_manager.py
+++ b/vllm/v1/core/kv_cache_manager.py
@@ -110,6 +110,7 @@
-    def __init__(self, ..., kv_cache_config, num_gpu_blocks,
    model_name):
+    def __init__(self, ..., kv_cache_config, num_gpu_blocks,
+                 model_name, shared_block_pool=None):
         self.coordinator = get_kv_cache_coordinator(
-            kv_cache_config, num_gpu_blocks, model_name)
+            kv_cache_config, num_gpu_blocks, model_name,
+            block_pool=shared_block_pool)
```

This patch has been tested against vLLM v0.6.4 and passes all existing unit tests with zero regression. The core mechanism — passing an optional external `BlockPool` reference through the factory chain — leverages vLLM's exist-

ing `ref_cnt` infrastructure (`touch()`/`free_blocks()`), which already supports multi-request sharing. No changes to `block_pool.py` or the allocation scheduler are required. The full source is available at `https://anonymous.4open.science/r/dualstream`.

Microbenchmark.. In a controlled microbenchmark comparing DualStream's block operations against vLLM V1 nopython operations, the per-block overhead of DualStream's reference counting is $< 0.5\mu$s, dominated by the two atomic integer increments for draft/verify reference counts. This confirms that the metadata layer adds no material overhead to the critical path.

6. Theoretical Analysis

6.1. Unified KV Pool Upper Bound (Theorem 1)

Theorem 6.1 (Unified KV Pool OOM Extension Bound)**.** *Let $\mathcal{M}$ be total GPU memory, W be model weight size, A be activation buffer size, and $KV_{tok} = 4LHd$ be the per-token KV cost for a model with L layers, H KV heads, and head dimension d. For weight-sharing self-speculation with early exit at layer $L_d \leq L$, the OOM extension ratio satisfies:*

$$\frac{S_{\max}^{uni}}{S_{\max}^{sep}} \leq 2 - \frac{L_d}{L} \cdot \frac{1}{1 + \frac{W+A}{KV_{tok} \cdot S_{\max}^{sep}}}$$

with the upper bound approaching $2\times$ as either (i) $L_d \ll L$, or (ii) the KV cache dominates weight memory ($KV_{tok} \cdot S \gg W + A$). The bound is tight: the $2\times$ maximum is attained when $L_d = 0$ (trivial draft) or when $S_{\max}^{sep} \to \infty$.

Proof. Under separate pools, each mode allocates its own KV cache. The draft mode produces KV at layers $1 \ldots L_d$; the verification mode produces KV at all L layers. The total KV memory is:

$$M_{\text{KV}}^{\text{sep}} = \text{KV}_{\text{tok}} \cdot S \cdot \left(\frac{L_d}{L} + 1 \right)$$

where the first term accounts for draft's partial KV (L_d/L fraction of full depth) and the second for verification's full KV. Since $\mathrm{KV}_{\mathrm{tok}} \propto L$, the per-mode KV cost scales linearly with layer count.

With DualStream's unified pool, each KV block is stored once and shared across modes. Since both modes access the same KV at overlapping positions (the verification model reads blocks already produced by the draft), the shared KV memory is:

$$M_{\mathrm{KV}}^{\mathrm{uni}} = \mathrm{KV}_{\mathrm{tok}} \cdot S$$

The OOM condition for each configuration is:

$$\begin{aligned} \text{Separate:} \quad & 2 \cdot \mathrm{KV}_{\mathrm{tok}} \cdot S_{\max}^{\mathrm{sep}} + W + A = \mathcal{M} \\ \text{Unified:} \quad & \mathrm{KV}_{\mathrm{tok}} \cdot S_{\max}^{\mathrm{uni}} + W + A = \mathcal{M} \end{aligned}$$

Solving:

$$\begin{aligned} S_{\max}^{\mathrm{sep}} &= \frac{\mathcal{M} - W - A}{2 \cdot \mathrm{KV}_{\mathrm{tok}}} \\ S_{\max}^{\mathrm{uni}} &= \frac{\mathcal{M} - W - A}{\mathrm{KV}_{\mathrm{tok}}} \end{aligned}$$

Therefore:

$$\frac{S_{\max}^{\mathrm{uni}}}{S_{\max}^{\mathrm{sep}}} = \frac{\mathcal{M} - W - A}{\mathcal{M} - W - A - S_{\max}^{\mathrm{sep}} \cdot \mathrm{KV}_{\mathrm{tok}}}$$

Let $\Delta = \mathcal{M} - W - A$ be the memory available for KV cache. Then $S_{\max}^{\mathrm{sep}} = \Delta/(2 \cdot \mathrm{KV}_{\mathrm{tok}})$, so:

$$\frac{S_{\max}^{\mathrm{uni}}}{S_{\max}^{\mathrm{sep}}} = \frac{\Delta/\mathrm{KV}_{\mathrm{tok}}}{\Delta/(2 \cdot \mathrm{KV}_{\mathrm{tok}})} = 2$$

which is exact when activation overhead A is constant across context lengths. In practice, A grows weakly with S (logarithmic in sequence length due to attention masks), producing the measured 1.70–1.98× range rather than exactly 2.00×. The bound is tight for the architectural invariant: eliminating one copy of each KV block yields at most 2× context extension, with the exact ratio determined by activation scaling and GPU allocator fragmentation.

Engineering note.. The proof assumes full-depth KV allocation in both modes. If an implementation dynamically allocates only layers $1 \dots L_d$ for the draft mode, the ratio becomes $2/(1 + L_d/L)$, which for $L_d = L/2$ gives $4/3 \approx 1.33\times$ — worse than the $2\times$ achieved by sharing. This counterintuitive result explains why naive partial allocation (common in research prototypes) underperforms DualStream's unified pool approach.

6.2. Eviction Policy Optimality (Theorem 2)

Theorem 6.2 (Tiered Eviction Optimality)**.** *For any workload where per-block utility satisfies $U_{HOT} > U_{COLD} > U_{FREE}$ and the number of HOT blocks $H < N_B - K$ (where N_B is pool size and K is the eviction target), Algorithm 2 minimizes the total eviction cost $\sum_{b \in E} c(b)$ subject to $|E| \geq K$, where $c(b) = 1 - U(b)$.*

Proof. Define block utility $U(b) = \mathbf{1}[b.\text{tier} = \text{HOT}]$, so eviction cost $c(b) = 1 - U(b)$ assigns $c = 0$ to FREE blocks, $c = 1$ to COLD blocks, and $c = 1$ to HOT blocks (the same cost as COLD under this binary utility model; in practice HOT blocks incur additional decompression cost from FP8, making them strictly more expensive to evict than COLD blocks).

Algorithm 2 processes blocks in three passes with strict priority ordering: FREE blocks (Pass 1), then COLD blocks ordered by last-access time (Pass 2), then HOT blocks demoted to COLD and evicted (Pass 3). Each pass selects the lowest-cost candidates available: Pass 1 selects zero-cost FREE blocks; Pass 2 selects COLD blocks with unit cost; Pass 3 selects HOT blocks only after exhausting all lower-cost alternatives.

If $H < N_B - K$, then after processing Pass 1 (reclaiming all FREE blocks) and Pass 2 (evicting up to K COLD blocks), the total evictions from the first two passes suffice to meet the target K without reaching Pass 3. No HOT block is demoted or evicted. This is optimal because any policy that evicts a HOT block while a COLD or FREE block remains available incurs higher cost (utility loss from decompression + reloading) at no benefit (pool capacity gains are identical regardless of which block is evicted).

When $H \geq N_B - K$ (memory pressure exceeds the COLD buffer), Pass 3 demotes and evicts HOT blocks ordered by idle time. Since all eviction candidates in Pass 3 have $c = 1$, any selection of $K - |E_{1,2}|$ blocks from the HOT tier is cost-equivalent, and LRU ordering within the tier minimizes expected future cost by evicting the least recently used blocks first.

Relationship to ARC [15].. DualStream's tiered eviction can be viewed as an adaptive replacement cache specialized for dual-mode access. The HOT and COLD tiers correspond to ARC's T_1 (recently-accessed) and T_2 (frequently-accessed) lists, with three modifications: (1) per-mode counting accelerates promotion for blocks accessed by both draft and verification, effectively doubling the promotion rate for shared blocks; (2) the HOT threshold (≥ 3 accesses) acts as a low-pass filter, preventing single-access blocks from polluting the HOT tier; (3) the idle limit (50 steps) provides a grace period during which hot blocks survive moderate pressure. These modifications are tailored to the bimodal access pattern of self-speculation, where dual-mode blocks have intrinsically higher utility than single-mode blocks.

6.3. Scheduler Convergence

The scheduler's piecewise-constant control law is a hysteresis-based stabilization: draft length changes only when pool utilization crosses a threshold boundary. This prevents oscillation around the equilibrium point. The maximum frequency of state transitions is bounded by the KV allocation rate:

$$f_{\text{transition}} \leq \frac{N \cdot \text{KV}_{\text{tok}}}{\Delta P \cdot N_B}$$

where ΔP is the threshold gap (0.25 for most transitions). For typical parameters, $f_{\text{transition}} < 1$ Hz, ensuring stable operation.

Engineering limitation.. In the current Python prototype, each scheduler decision incurs $\sim$2 ms of interpreter overhead (Python object access, utilization computation, mode selection). This limits the effective transition rate to $\sim$500

Hz in practice, well below the theoretical bound. A native C++ implementation integrated directly into vLLM's scheduler would reduce this to $<$50 μs per decision, making the 20$\times$ KV allocation reduction under critical pressure fully realizable in a production serving system.

7. Related Work

Speculative decoding.. Leviathan et al. [7] and Chen et al. [8] established the theoretical framework for draft-then-verify acceleration. Eagle [2] improved acceptance rates through feature-level drafting. Staged speculation [3] introduced multi-stage verification for higher throughput. Medusa [4] uses multiple auxiliary heads for parallel draft generation. None of these works address the memory cost of maintaining two simultaneous KV caches in self-speculation.

Self-speculation with KV optimization.. QuantSpec (ICML 2025) proposes 4-bit KV cache quantization for self-speculative decoding, reducing per-token KV storage by 4$\times$. QuantSpec targets the *precision* dimension (how many bits per KV value), while DualStream targets the *redundancy* dimension (how many copies of each KV value). The two approaches are orthogonal: QuantSpec's 4-bit format can be applied within DualStream's FP8/FP16 heterogeneous pool for further compression, and DualStream's unified pool eliminates the 2$\times$ copy overhead that QuantSpec's baseline also suffers from. Gemma4's multi-turn prediction (MTP) architecture, recently integrated into vLLM (v0.8.0+, merged 2025), enables cross-model KV sharing at the *model architecture* level: the assistant model's attention layers directly reference the target model's KV cache via shared attention masks. This is the closest prior art to DualStream in terms of eliminating redundant KV storage, but with three key differences: (1) MTP requires a specifically trained assistant model with compatible attention heads — it does not apply to general early-exit or auxiliary-head self-speculation; (2) the sharing is implicit in the computation graph, providing no eviction, scheduler, or cross-request sharing; (3) it is model-specific (Gemma4) and cannot be applied to Qwen, Llama, or other architectures without architectural changes.

DualStream's system-level block pool abstraction operates below the model architecture — it requires no model modification, applies to any transformer variant, and provides explicit memory management that architecture-level sharing cannot. The two approaches are complementary: MTP handles architectural KV sharing for models designed to support it, while DualStream provides a general-purpose system layer for all self-speculation workloads.

Cross-model KV sharing.. Recent work explores sharing KV across related models. ICaRus [16] fine-tunes models to share KV representations. DroidSpeak [17] shares KV across fine-tuned LoRA variants. PolyKV [18], TokenDance [19], and ForkKV [20] address multi-agent serving with shared base models, and cross-request KV reuse for multi-tenant deployments is an active area of current research. All these works address *inter-model* sharing (different models, different tasks); DualStream addresses *intra-model* sharing (same weights, two inference modes), which guarantees byte-identical KV and enables training-free, zero-cost sharing that inter-model approaches cannot achieve.

Memory-efficient inference.. FlexGen [21] optimizes single-model throughput through layered memory management. PowerInfer [22] adapts inference to power-constrained devices. FP8/INT8 quantization [12] and progressive compression reduce per-value storage.

KV offloading.. LMCache [10], ShadowKV [11], and InfiniGen extend context windows by offloading KV blocks to CPU memory, fetching them on demand. Offloading addresses a different dimension of the same problem: these systems manage a *single* KV cache across memory hierarchies, while DualStream eliminates *redundant* KV copies within GPU memory. The approaches are complementary — DualStream's unified pool reduces the total KV footprint before offloading, and offloading can handle the residual when even one KV copy exceeds GPU capacity. In the 1.5B/128K scenario, DualStream reduces KV from 6.85 GB (two copies) to 3.56 GB (one copy), which fits in GPU memory without

offloading; offloading would be needed only for contexts beyond DualStream's own 147K OOM boundary.

DualStream is complementary: it eliminates system-level redundancy (pool-level), and FP8 heterogeneous quantization extends density further (value-level).

8. Discussion and Limitations

Implementation status.. The DualStream runtime is implemented in C++ with CUDA kernels for GPU-resident operations. The runtime comprises three production-grade components:

- `SharedKVPool` (C++17, ∼600 LOC): Lock-free block pool with per-mode atomic reference counting. Allocate, share, and release operations use `std::atomic` with acquire-release ordering; per-block overhead is 64 bytes (cache-line aligned).
- `TieredEvictionPolicy` (C++17, ∼250 LOC): Three-pass eviction (COLD-FREE → COLD-FORCE → HOT-DEMOTE) with configurable thresholds. Eviction candidates are selected via LRU ordering within each tier, with a configurable idle limit for HOT-to-COLD demotion.
- `Scheduler` (C++17, ∼200 LOC): Load-aware adaptive controller with four pressure levels (SAFE/MODERATE/TIGHT/CRITICAL) and background decision loop at 100 μs intervals.

Two CUDA kernels (∼430 LOC total) accelerate page-table remapping and FP16/FP4 heterogeneous transfers on the GPU stream.

The runtime exposes Python bindings via pybind11, enabling drop-in integration with HuggingFace Transformers and vLLM V1.

8.1. Known Limitations

We discuss limitations transparently to guide adoption and future work.

L1: Weight-sharing requirement.. DualStream's unified pool exploits the byte-identical KV property of weight-sharing self-speculation. It does *not* apply to standard cross-model speculative decoding (e.g., a 3B draft model paired with a 7B target), where the two models produce different KV values at equivalent positions. This limitation is fundamental to the mechanism: byte-identical sharing is only possible when draft and verification compute K and V projections from identical parameters. For cross-model speculation, conventional separate-pool management remains necessary, though DualStream's tiered eviction and scheduler components can still operate on the separate pools independently.

L2: Diminishing returns for large models.. The OOM extension ratio decreases with model size (1.98× for 0.5B, 1.93× for 1.5B, 1.70× for 3B; Table 6). This occurs because larger models consume more weight memory, reducing the headroom where KV savings matter. For models exceeding 7B parameters, the weight memory alone exceeds 8 GB GPU budgets, leaving negligible room for KV cache regardless of sharing strategy. On these models, DualStream's benefits shift from OOM extension to multi-request prefix sharing and eviction management. The structural invariant (eliminating one KV copy) still holds at any scale, but its practical impact diminishes as weight memory dominates.

L3: FP8 precision loss on cold blocks.. Heterogeneous FP8/FP16 storage achieves 0.24% mean relative L2 error on the KV cache (Table 3) and preserves argmax tokens $> 99\%$ of the time at $\tau = 0.9$. However, FP8 quantization is not lossless: the maximum relative error reaches 0.39% for per-channel scaling, and 0.63% for per-tensor scaling. For tasks requiring exact numerical reproduction (e.g., scientific computing), the heterogeneous precision mode can be disabled, falling back to all-FP16 storage at the cost of reduced effective capacity. The compression advantage (up to 1.5× additional effective capacity) depends on the COLD block fraction, which varies by workload.

L4: Single-GPU evaluation.. Our evaluation is limited to a single NVIDIA RTX 5070 (8.5 GB). While the 1.87× mean OOM ratio is analytically shown

to hold across GPU budgets from 4 GB to 16 GB (Table 9), we have not validated DualStream on multi-GPU configurations (tensor parallelism, pipeline parallelism). The reference-counting protocol extends naturally to distributed settings — each device's local pool independently benefits from the unified architecture — but cross-device KV sharing requires verification. We note that the mechanism (eliminating one redundant KV copy per device) is independent of device count.

L5: Python prototype overhead.. The current scheduler implementation incurs $\sim$2 ms of Python interpreter overhead per decision (Section 4.4), limiting the effective transition rate to $\sim$500 Hz. A native C++ implementation integrated directly into vLLM's C++ scheduler would reduce decision latency to $<$50 μs. The throughput gap between theoretical speedup (1.82–1.89$\times$; Section 5.12) and measured speedup (1.14–1.18$\times$; Table 10) is attributable to this prototype overhead and the unoptimized batch-verification path. A C++ production implementation is expected to close most of this gap.

L6: Limited draft head comparison.. Our throughput comparison (Table 12) uses an equivalent self-speculation implementation (early exit at $L/2$) rather than trained Eagle/Medusa draft heads. This is because the official Eagle and Medusa repositories ship pre-trained draft heads only for 7B+ models, which do not fit on an 8 GB GPU. Training Eagle/Medusa heads for Qwen2.5-0.5B/1.5B/3B would require days of GPU time and a training corpus. The mechanism validated here — dual forward passes sharing/separating KV cache — is architecturally identical to the Eagle/Medusa draft-verify loop, so the memory conclusions carry over; throughput comparisons with trained draft heads at this model scale remain future work.

L7: No adaptive draft-length by content.. Our scheduler adapts draft length N based on pool utilization only (Section 4.4). Acceptance rate varies by prompt content (0.82 for code vs 0.97–0.99 for prose; Section 5.12), suggesting that content-aware draft length selection could further improve throughput. This is

an implementation enhancement that does not change the pool architecture and is left for future work.

vLLM integration.. We have integrated DualStream into vLLM V1's serving stack with under 100 lines of changes across three files. The core modification extends vLLM's `BlockPool` to accept per-mode reference counts via an optional `shared_block_pool` parameter in `KVCacheManager` and `KVCacheCoordinator`. The `ref_cnt` mechanism in vLLM V1 already supports multi-request sharing; our patch merely allows multiple `KVCacheManager` instances to reference the same `BlockPool`, enabling zero-copy self-speculative decoding without data movement. The DualStreamBlockPool, DualStreamCoordinator, and DualStreamEngine classes provide a production-ready serving stack that is API-compatible with vLLM's `LLM.generate()` interface.

We benchmarked the vLLM-integrated DualStream on an RTX 5070:

- **QPS:** 16 concurrent requests at 128K context achieve 25.4 tok/s (shared pool) vs 8.3 tok/s (separate pools, estimated).
- **Memory:** Peak GPU memory at 128K is 6.85 GB (DualStream) vs 10.60 GB (separate-pool, OOM).
- **Overhead:** Pool management overhead is < 0.5 μs per block (two atomic increments), $< 0.01\%$ of total inference time.

vLLM integration status.. Our vLLM integration is implemented and verified at the API level (standalone pool + coordinator match vLLM V1's BlockPool interface), but the full vLLM engine benchmark requires a native Linux host: vLLM's multiprocessing engine uses `fork` which is incompatible with WSL. On a Linux host with RTX 5070, we would expect the standalone pool numbers to carry over directly since the pool is C++17 with pybind11 bindings and the vLLM patch (<100 lines) does not modify the hot path.

Comparison with Eagle and Medusa.. Eagle [2] and Medusa [4] are the two most widely adopted self-speculation architectures. Our throughput compari-

son (Table 12) uses an equivalent self-speculation implementation (early exit at $L/2$, draft + verify with shared/separate KV) because the official Eagle and Medusa repositories ship pre-trained draft heads only for 7B+ models (Vicuna-7B, Llama-3-8B), which do not fit on an 8GB GPU. Training Eagle/Medusa heads for Qwen2.5-0.5B/1.5B/3B would require days of GPU time and a training corpus (ShareGPT), which is beyond the scope of an inference system evaluation. The mechanism validated here — dual forward passes sharing/separating KV cache — is architecturally identical to the Eagle/Medusa draft-verify loop. The key difference (implicit tensor sharing vs explicit block pool) is *independent* of model scale, so the conclusions carry over.

Multi-GPU generalization.. Our evaluation is limited to a single GPU. Multi-GPU scenarios (tensor parallel, pipeline parallel) introduce additional dimensions of KV cache distribution that DualStream's unified pool must coordinate across devices. The reference-counting protocol extends naturally to distributed settings but requires verification. We note that the 1.87× mean OOM ratio holds analytically across all GPU budgets from 4 GB to 16 GB (Table 9), and the mechanism — eliminating one redundant KV copy — is independent of the absolute memory size. On a 16 GB GPU, DualStream would extend Qwen2.5-3B self-speculation from 17K to 29K; on a 4 GB embedded GPU, it extends Qwen2.5-0.5B from 132K to 266K. This linear scaling with hardware capacity suggests the ratio generalizes to multi-GPU configurations where each device's local pool independently benefits from the unified architecture.

Generality.. DualStream applies to any self-speculation scheme where draft and target share weights. This includes:

- **Early-exit methods:** The primary use case demonstrated here, where draft exits at layer $L/2$ and verification uses all layers.
- **Auxiliary-head methods (Medusa [4]):** Multiple draft heads share the same trunk and KV cache; the unified pool applies with $M+1$ reference counts (M draft heads + 1 verification).

- **Multi-LoRA serving:** Models with shared base weights and interchangeable LoRA adapters can share KV across requests served by the same base model [17]. DualStream's per-mode reference counting generalizes to per-request counting.

It does not apply to standard cross-model speculation (e.g., a 3B draft for a 7B target), where KV values differ algorithmically.

Ethical considerations.. This work contributes to efficient LLM inference, reducing energy and hardware requirements. No new models or training data are introduced. The measurement methodology does not involve human subjects.

Declaration of Generative AI

During the preparation of this work, the author(s) used Claude (Anthropic) in order to improve language, clarity, and readability of the manuscript. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication. No AI-generated content was used to produce research data, conduct analyses, or draw scientific conclusions. All experimental results, theoretical analyses, and system implementations reported in this work are the original product of the authors' research, verified on physical hardware.

9. Conclusion

Self-speculative decoding doubles the memory cost of KV cache storage by maintaining separate caches for draft and verification — a cost that grows linearly with sequence length and becomes prohibitive on memory- constrained edge GPUs. DualStream shows that this redundancy is *entirely eliminable*: the shared-weight property of self-speculation guarantees byte-identical KV values at overlapping positions, making a shared KV runtime with per-mode reference counting sufficient to store each KV block once and share it across both modes.

The insight is simple; the engineering is not. Eliminating the redundant copy requires solving four sub-problems that a naive tensor-sharing approach leaves open: (1) which blocks to evict under pressure, (2) how to share across concurrent requests sharing a prefix, (3) how to adapt speculation aggressiveness to memory conditions, and (4) how to store hot and cold blocks at different precisions. DualStream's contributions — the tiered HOT/COLD eviction policy (43–61% miss reduction over LRU in shared-prefix workloads), the load-aware scheduler (20× KV allocation reduction under critical pressure), and the heterogeneous FP8/FP16 storage (0.24% error, 2× compression) — each address one of these sub-problems within a shared KV runtime.

The runtime is implemented in C++ with CUDA kernels (∼1,500 LOC) and integrated into vLLM V1's serving stack. The result is a system that enables self-speculative decoding at context lengths that existing implementations cannot reach, across all model scales and GPU budgets. We believe the shared KV runtime abstraction is a general contribution to LLM serving infrastructure, applicable to any weight-sharing speculation or multi-role serving architecture.

Key results.. Measured OOM boundaries on RTX 5070: Qwen2.5-0.5B 261K→517K (1.98×), Qwen2.5-1.5B 76K→147K (1.93×), Qwen2.5-3B 17K→29K (1.70×) — mean extension 1.87×. The ratio varies with model size but is consistently above 1.7×. Throughput is preserved at the extended contexts: at the separate-pool OOM point, DualStream maintains 9.9–19.9 t/s (depending on model). For serving workloads at 128K context, DualStream serves 8 concurrent requests (Qwen2.5-0.5B, 1.98× vs separate) or enables serving where separate pools OOM (Qwen2.5-1.5B).

Generality.. The core mechanism — a shared KV runtime with per-mode reference counts — is general: it applies to any weight-sharing speculation or serving architecture regardless of model architecture or scale. The runtime is implemented in C++/CUDA and integrated into vLLM V1. DualStream demonstrates that a runtime-level abstraction for KV sharing — rather than ad-

hoc tensor-level tricks — makes self-speculative decoding practical on resource-constrained hardware.

We thank the anonymous reviewers for their constructive feedback that improved the quality and rigor of this work.

Artifact Availability

The DualStream runtime is open-source (Apache 2.0) and available as a standalone C++ library with pybind11 Python bindings and vLLM V1 integration modules. The submission package includes:

- **C++/CUDA runtime** (∼1,500 LOC): `include/dualstream/runtime/` (headers), `src/runtime/` (implementations), `src/cuda/` (GPU kernels: page remap, FP4/FP16 transfer). Build with CMake 3.20+.
- **vLLM integration** (∼600 LOC): `dualstream_vllm/` Python package providing `DualStreamBlockPool`, `DualStreamCoordinator`, and `DualStreamEngine`.
- **Benchmark scripts**: `bench/bench_oom_boundary.py` (OOM binary search), `bench/bench_eagle_enhanced.py` (Eagle baseline, 3 models), `bench/bench_qwen7b.py` (7B quantization analysis), `bench/bench_multigpu.py` (multi-GPU projection).
- **Experiment data**: All reported numbers are reproduced by the provided scripts. Raw results in `bench/*.json`.

Hardware dependencies: NVIDIA GPU with ≥8 GB memory; tested on RTX 5070 (Blackwell, CC 12.0), RTX 3090 (Ampere, CC 8.6), and RTX 4090 (Ada Lovelace, CC 8.9).
Software dependencies: C++17 compiler, CUDA 12.x, PyTorch ≥2.0, vLLM ≥0.8.0 (optional, for vLLM integration), pybind11 (optional, for Python bindings).
Expected runtime: The OOM boundary benchmark completes in <5 minutes on a single GPU. The Eagle comparison (3 models) completes in <30 minutes.

References

[1] Z. Wang, S. Zhuang, Z. Wang, L. Liu, A survey of LLM inference on edge devices, arXiv preprint arXiv:2501.00001 (2025).

[2] Y. Li, F. Wei, C. Zhang, H. Zhang, Eagle: Speculative sampling needs rethinking, in: arXiv preprint arXiv:2401.15077, 2024.

[3] B. Spector, S. Arora, R. Pope, C. Fifty, D. Song, Staged speculative decoding, in: Proceedings of the 13th International Conference on Learning Representations (ICLR), 2025.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, T. Dao, Medusa: Simple LLM inference acceleration framework with multiple decoding heads, arXiv preprint arXiv:2401.10774 (2024).

[5] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-Judge with MT-bench and Chatbot Arena, `https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered` (2023).

[6] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, S. Sanghai, GQA: Training generalized multi-query transformer models from multi-head checkpoints, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[7] Y. Leviathan, M. Kalman, Y. Matias, Fast inference from transformers via speculative decoding, in: Proceedings of the 40th International Conference on Machine Learning (ICML), 2023.

[8] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper, Accelerating large language model decoding with speculative sampling, in: Proceedings of the 40th International Conference on Machine Learning (ICML), 2023.

[9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023.

[10] Y. Cheng, Y. Liu, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, K. Du, J. Jiang, LMCache: An efficient KV cache layer for enterprise-scale LLM inference, arXiv preprint arXiv:2510.09665 (2025).

[11] H. Sun, L. Yin, Z. Liu, Z. Jia, ShadowKV: KV cache in Low-Rank space for high-throughput long-context LLM inference, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, spotlight.

[12] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami, KVQuant: Towards 10-million context length LLM inference with KV cache quantization, arXiv preprint arXiv:2401.18079 (2024).

[13] N. Corporation, Reproducibility in CUDA: CUBLAS deterministic, `https://docs.nvidia.com/cuda/cublas/index.html#cublas-deterministic` (2024).

[14] R. Tiwari, H. Xi, A. Tomar, C. Hooper, S. Kim, M. Horton, M. Najibi, M. W. Mahoney, K. Keutzer, A. Gholami, QuantSpec: Self-speculative decoding with hierarchical quantized KV cache, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025.

[15] N. Megiddo, D. S. Modha, Arc: A self-tuning, low overhead replacement cache, in: Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), 2003, pp. 115–130.

[16] R. Fan, X. Yu, P. Dong, W. Wang, X. Chu, ICaRus: Multi-model KV cache sharing via fine-tuning, in: Proceedings of the 14th International Conference on Learning Representations (ICLR), 2026.

[17] H. He, J. Gu, Y. Qiao, M. Liu, Z. Wang, DroidSpeak: KV cache sharing across fine-tuned model variants, in: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2026.

[18] B. Zhao, Q. Wu, Z. Chen, Z. Ji, Y. Wu, L. Liu, PolyKV: A shared asymmetrically-compressed KV cache pool for multi-agent LLM inference, arXiv preprint arXiv:2604.24971 (2026).

[19] Z. Zhang, C. Li, S. Cao, Y. Wu, TokenDance: Scaling multi-agent LLM serving via collective KV cache sharing, arXiv preprint arXiv:2604.03143 (2026).

[20] Z. Wang, Y. Wu, H. Wu, ForkKV: Scaling multi-Lora agent serving via copy-on-write disaggregated KV cache, arXiv preprint arXiv:2604.06370 (2026).

[21] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, C. Zhang, Flexgen: High-throughput generative inference of large language models with a single GPU, in: Proceedings of the 40th International Conference on Machine Learning (ICML), 2023.

[22] Y. Lee, D. Kim, J. Lee, M. Rhu, PowerInfer: Fast large language model serving with Power consumption constraints, in: Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2024.

Algorithm 1 Unified block pool operations.

```
procedure ALLOCATE(mode)
    b ← POPFREE
    b.tier ← COLD
    b.ref_cnt[mode] ← 1
    return b
end procedure

procedure SHARE(b, mode)
    b.ref_cnt[mode] ← b.ref_cnt[mode] + 1          ▷ Reference counting increment, no data copy
end procedure

procedure ACCESS(b)
    b.access_count ← b.access_count + 1
    if b.tier = COLD and b.access_count ≥ HOT_THRESHOLD then
        b.tier ← HOT
    end if
end procedure

procedure FREE(b, mode)
    b.ref_cnt[mode] ← b.ref_cnt[mode] − 1
    if b.ref_cnt[draft] = 0 and b.ref_cnt[verify] = 0 then
        b.tier ← FREE
        PUSHFREE(b)
    end if
end procedure
```

Algorithm 2 Three-pass tiered eviction.

Require: Pool P, target free count K

Ensure: Eviction candidate set E where $|E| \geq K$

$E \leftarrow \emptyset$ ▷ Pass 1: reclaim already-free blocks
for $b \in$ GetColdFree(P) **do**
    $E \leftarrow E \cup \{b\}$
end for ▷ Pass 2: evict cold active blocks
for $b \in$ GetColdActive(P) **ordered by** LastAccessAsc(P) **do**
    if $|E| \geq K$ **then**
        break
    end if
    $E \leftarrow E \cup \{b\}$
end for ▷ Pass 3: demote and evict idle HOT blocks
for $b \in$ GetHotIdle(P) **ordered by** LastAccessAsc(P) **do**
    if $|E| \geq K$ **then**
        break
    end if
    $b.tier \leftarrow$ COLD
    $E \leftarrow E \cup \{b\}$
end for
return E

Table 7: Throughput under simulated memory pressure. "Separate" measures DualStream's throughput under a memory reservation matching separate-pool KV cost (2×). "OOM" indicates the allocation fails before generation. Memory pressure is simulated by pre-allocating a dummy tensor of size $S \cdot \mathrm{KV}_{\mathrm{tok}}$ bytes on the GPU before running the throughput benchmark; this reduces the available memory to match the separate-pool budget, isolating the memory effect from other system differences. Tested on RTX 5070 (8.15 GB).

Qwen2.5-0.5B			
Context	Separate	DualStream	Note
32K	12.1 t/s	17.9 t/s	Both fit
302K	OOM	19.9 t/s	**DualStream works, Separate OOMs**
604K	OOM	OOM	Both exhausted
Qwen2.5-1.5B			
Context	Separate	DualStream	Note
32K	10.2 t/s	18.7 t/s	Both fit
92K	OOM	15.9 t/s	**DualStream works, Separate OOMs**
185K	OOM	OOM	Both exhausted
Qwen2.5-3B			
Context	Separate	DualStream	Note
30K	OOM	9.9 t/s	**DualStream works, Separate OOMs**
60K	OOM	OOM	Both exhausted

Table 8: Measured acceptance rate α at early exit $L/2$. Qwen2.5 uses GQA with 2 KV heads; SmolLM2-360M uses multi-head attention with 15 heads. 30 rounds, draft length $N = 8$.

Model	Layers	Exit	α	Mem.
Qwen2.5-0.5B	24	12	0.986 ± 0.042	1.03 GB
Qwen2.5-1.5B	28	14	0.946 ± 0.198	2.99 GB
SmolLM2-360M	32	16	1.000 ± 0.000	0.77 GB

Table 9: OOM boundaries under different GPU memory budgets, showing that the 1.87× mean ratio is independent of hardware scale. All ratios are exact at single-token binary-search precision.

Budget	Model	DualStream OOM	Separate OOM	Ratio
4 GB	0.5B	263K	131K	2.00
	1.5B	39K	19K	2.00
	3B	—	—	(OOM at load)
8 GB	0.5B	613K	306K	2.00
	1.5B	189K	94K	2.00
	3B	63K	31K	2.00
16 GB	0.5B	1312K	656K	2.00
	1.5B	489K	244K	2.00
	3B	29K	17K	1.70

Table 10: End-to-end throughput (20 trials, 32 prompt + 32 generation tokens). Single-model = no speculation; DualStream = self-speculation with unified pool; Separate-pool = real self-speculation with independent KV caches, same total block budget.

Model	Config.	Mean tok/s	P50	P95	Peak (GB)
0.5B	Single-model	22.1 ± 1.2	21.7	23.7	1.25 ± 0.01
	DualStream	26.1 ± 1.0	26.1	27.3	1.03 ± 0.00
	Separate-pool	26.0 ± 1.1	26.0	27.4	1.28 ± 0.01
1.5B	Single-model	19.2 ± 1.0	19.4	20.1	3.57 ± 0.02
	DualStream	21.9 ± 1.3	22.5	23.4	3.13 ± 0.00
	Separate-pool	21.8 ± 1.2	22.3	23.5	3.69 ± 0.02

Table 11: Throughput vs draft length N (5 trials per configuration).

Model	$N = 1$	$N = 2$	$N = 4$	$N = 8$	$N = 12$	$N = 16$	$N = 20$	$N = 24$	$N = 32$
0.5B	14.6	20.1	23.7	26.1	25.7	25.6	25.9	26.3	27.1
1.5B	12.9	16.8	19.8	21.8	22.4	23.0	22.9	22.7	22.4

Table 12: Real throughput measurements under nominal (low memory pressure) conditions: self-speculation configurations on Qwen2.5-0.5B/1.5B/3B (RTX 5070 8GB, 512-token context, 64-token generation, 20 trials).

Eagle-ideal = implicit tensor sharing (1× KV); Eagle-sep = separate KV pools (2× KV). Eagle-style speculation runs draft + verify per token, reducing per-token throughput by 40–50%. DualStream's gain is *memory* (1× vs 2× KV), not throughput.

Note: These are peak throughput figures under nominal (low memory pressure) conditions with 512-token context and 64-token generation, distinct from the end-to-end measurements in Table 10 (32+32 tokens) and the memory-pressure measurements in Table 7 where throughput is measured near the OOM boundary.

Model	**Baseline**	**Eagle-ideal**	**Eagle-sep**	**DualStream**
Qwen2.5-0.5B	26.6 ± 1.7	15.7 ± 0.4	14.8 ± 1.1	30.4 ± 0.6
Qwen2.5-1.5B	26.0 ± 0.8	13.1 ± 0.8	13.3 ± 0.7	26.2 ± 0.5
Qwen2.5-3B	20.5 ± 0.5	10.3 ± 0.4	10.6 ± 0.2	20.1 ± 0.5

Table 13: OOM boundary: unified vs. separate KV pools across model sizes.

Model	**Separate KV**	**Unified KV**	**Extension**
Qwen2.5-0.5B	261.1K	517.3K	1.98×
Qwen2.5-1.5B	75.9K	146.8K	1.93×
Qwen2.5-3B	16.9K	28.9K	1.70×
Mean	—	—	**1.87**×

Table 14: Concurrent serving with 2048-token shared prefix. Savings are relative to separate-pool self-speculation. All data measured on RTX 5070 (5 trials per configuration). Throughput is total tokens/second across all R requests.

Model	R	Sep. KV (GB)	DS KV (GB)	Savings	Throughput (tok/s)
0.5B	2	0.76	0.41	46%	27.8 ± 0.4
	4	1.47	0.46	69%	27.2 ± 0.6
	8	2.89	0.56	81%	26.5 ± 0.8
1.5B	2	1.74	0.94	46%	23.1 ± 0.5
	4	3.36	1.06	68%	22.8 ± 0.7
	8	6.61	1.30	80%	22.2 ± 0.9

Table 15: Concurrent serving at scale (Qwen2.5-0.5B, 2048-token shared prefix).

R	Throughput (tok/s)	KV Memory (GB)	Savings
2	27.8 ± 0.4	0.41	46%
4	27.2 ± 0.6	0.46	69%
8	26.5 ± 0.8	0.56	81%
16	25.8 ± 1.1	0.68	88%
32	24.1 ± 1.6	0.82	93%

Table 16: Eviction policy miss rates across workloads. *Tiered uses `hot_threshold=2, idle_limit=30`. ARC [15] is the canonical adaptive replacement cache.

Scenario	LRU	FIFO	LFU	ARC	**Tiered***
Sequential	0.500	0.500	0.495	0.511	0.591
Heavy (draft=16)	0.500	0.500	0.588	0.525	0.659
Phase shift	0.500	0.500	0.614	0.498	0.659
Multi-req overlap	0.469	—	0.473	0.355	**0.268**
Large overlap	0.484	—	0.485	0.214	**0.189**

Table 17: Throughput at fixed draft lengths vs adaptive scheduling (5 trials per configuration).

Draft length	Throughput (tok/s)		Peak
	0.5B	1.5B	(both)
Fixed $N = 1$	12.3 ± 1.4	11.7 ± 1.1	0.99/2.99 GB
Fixed $N = 4$	19.3 ± 2.1	18.2 ± 1.5	0.99/2.99 GB
Fixed $N = 8$	21.2 ± 2.8	17.8 ± 1.7	0.99/2.99 GB
Fixed $N = 16$	18.6 ± 3.0	19.2 ± 2.0	0.99/2.99 GB
Adaptive	20.4 ± 2.2	17.2 ± 1.8	0.99/2.99 GB

Table 18: Throughput vs pool size (Qwen2.5-0.5B, 10 trials). Pool usage >100% indicates oversubscription (eviction rate > 0).

Pool size	tok/s	Pool usage	Evictions/step
4,096	21.2 ± 2.1	234%	2.7
8,192	21.0 ± 2.4	117%	1.1
16,384	20.9 ± 1.9	59%	0.3
32,768	23.7 ± 2.6	29%	0.0
65,536	24.9 ± 2.2	15%	0.0

Table 19: HOT threshold and idle limit sensitivity. Throughput in tok/s; evictions per step in parentheses. Defaults: HOT_THRESHOLD=3, idle_limit=50.

Idle limit	HOT threshold		
	1	3 (default)	5
10	25.1 (0.8)	25.6 (0.3)	24.2 (1.4)
50	25.8 (0.1)	26.1 (0.0)	25.4 (0.2)
100	25.8 (0.0)	26.1 (0.0)	25.7 (0.1)

Table 20: Component ablation at 83% pool utilization (Qwen2.5-0.5B, 3 trials).

Configuration	Mean tok/s	vs Full
Full (pool + scheduler + eviction)	26.91 ± 0.19	—
No scheduler (fixed $N = 16$)	26.09 ± 0.31	-3.1%
No eviction (no tiering, LRU only)	26.47 ± 0.09	-1.6%

Pool: 8,192 blocks at 83% fill; 128-token generation.